



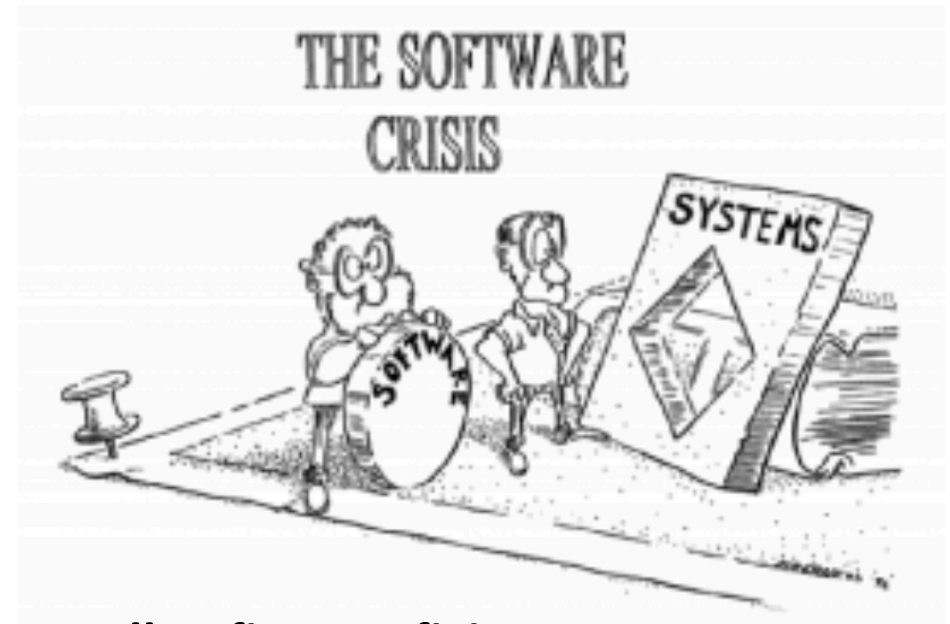
Unidad 1 – Arquitectura de Sistemas

Arquitectura de Sistemas

Lic. Marcelo Palma

mpalma@uch.edu.ar
marcelo.palma.a@gmail.com

Arquitectura del software

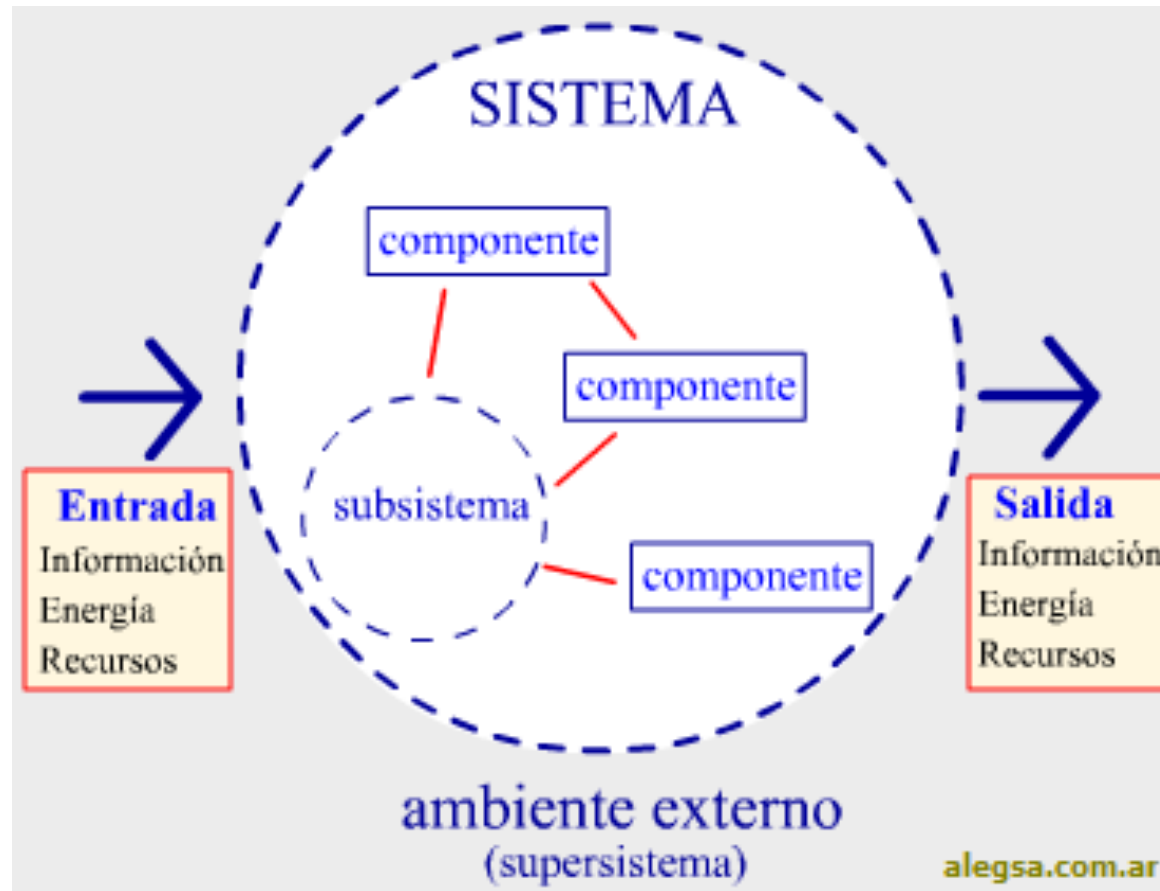


1. **Porqué?** **Sistemas complejos ...**
2. **Para qué?** **Calidad del software ... desarrollo eficaz y eficiente ...**
3. **Cuándo?** **1969 ... 1990 década de la arquitectura del sw ...**
4. **Donde?** **NATO ... Ingeniería de Software ... The Humble Programmer**
5. **Quién?** **Edsger Dijkstra! Comunidad científica, académica, empresarial**



Conceptos previos

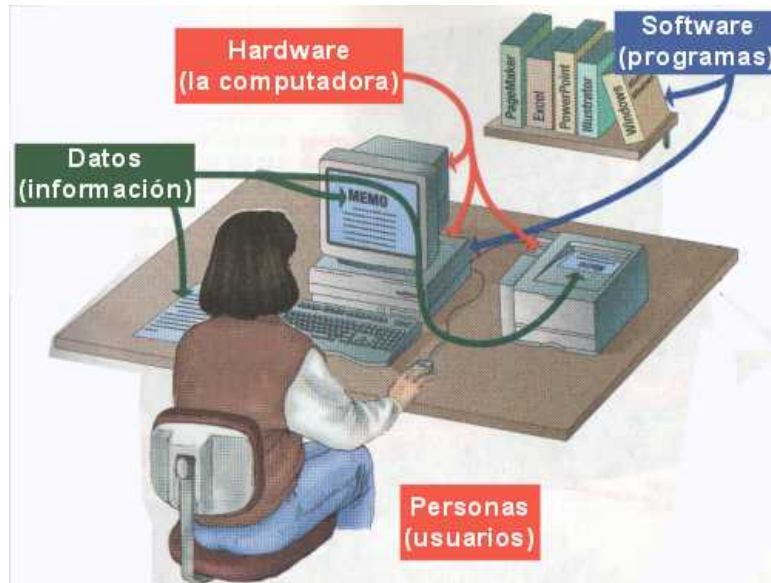
Qué es un “Sistema”



“Un **Sistema** es un conjunto de elementos vinculados entre sí, que se comportan como un todo, tienen una frontera bien definida y están organizados para lograr un fin determinado.”

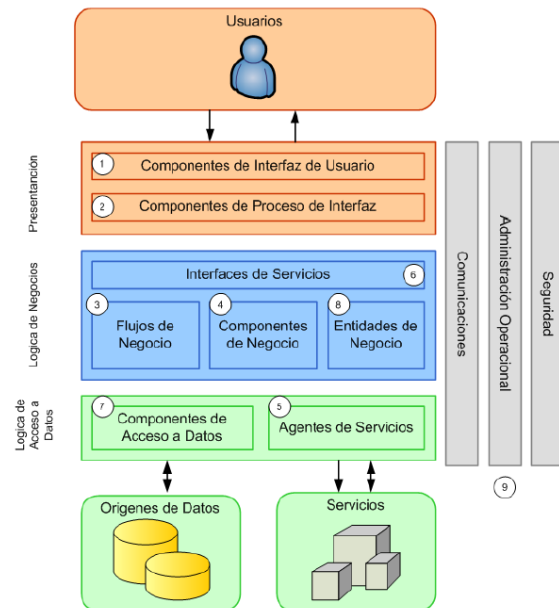
Qué es un “Sistema de Computación”

“Un sistema de computación es un conjunto de elementos informáticos, vinculados entre sí, para llevar a cabo algún método, procedimiento o control mediante el procesamiento de información.”



“Sistemas intensivos en software”

Qué es un “Sistema de software”



“Un sistema de software es un conjunto de elementos software, vinculados entre sí, para llevar a cabo algún método, procedimiento o control mediante el procesamiento de información.”

Reseña histórica

1968: Edsger Dijkstra

- Estructura antes de programar.
- Niveles de Abstracción.
- Diseño Conceptual – Niklaus Wirth

1969: P.I. Sharp formula

- Arquitectura del Software.
- Especificaciones: Funcionales + Diseño + Forma.

1970: Diseño Estructurado

- Modelos de desarrollo de software.
- Salen de cascada a modelos Orgánicos, Evolutivos y Cíclicos.

1972 - 1976: David Parnas

1975: Brooks

- Módulos y Ocultamiento de Información.
- Estructuras de software y Familias de Programas.
- Búsqueda de Calidad del Software.
- Decisiones de Diseño Tempranas.

- Arquitectura como la especificación completa de la IU.
- Arquitectura (¿Qué?) vs. Implementación (¿Cómo?)

Reseña histórica II

1960

La historia de la arquitectura del software remonta a la época de los años 60 se empieza a utilizar el término cuando Edsger Dijkstra la Universidad Tecnológica de Eindhoven en Holanda en 1968 propuso que se establezca una estructura correcta de los sistemas de software antes de que se inicie la programación como tal, escribiendo código de cualquier manera.

1969

un año después de la sesión en que se fundara la ingeniería de software, P. I. Sharp formuló que la ingeniería era diferente a la “arquitectura”.

Fred Brooks Jr y Ken Iverson llamaban arquitectura a la estructura conceptual de un sistema en la perspectiva del programador

1970

En 1972, Parnas publicó un ensayo en el que discutía la forma en que la modularidad en el diseño de sistemas podía mejorar la flexibilidad y el control conceptual del sistema, introduciendo el concepto de Ocultamiento de información, La herencia de este concepto en la ingeniería y la arquitectura ulterior es inmensa, y se confunde estrechamente con la idea de abstracción.

1975

Brooks, diseñador del sistema operativo OS/360 y Premio Turing 2000, utilizaba el concepto de arquitectura del sistema para designar “la especificación completa y detallada de la interfaz de usuario”.

Reseña histórica III

1980s: POO

- 1960 (Simula) TADs y Clases, Smalltalk
- Teoría: Modelar el Dominio del problema e implementarlo en un lenguaje POO.

1992: Perry & Wolf

- Arquitectura del Software análogo a la construcción de edificios. Bases de la AS (Elementos, Forma, Razón).
- Herramientas CASE.

1990s: Apogeo de la AS

2000: Roy Fielding

- Modelo REST: Tecnologías Web y Modelo Orientado a Servicios.
- IEEE Std 1471.

- 1996 - Paul Clements: Componentes.
- 1995 – Patrones de Diseño. B4.
- Estilos arquitectónicos (ADLs)
- Vistas Arquitectónicas. Modelo propuestos:
 - 4+1.
 - TOGAF.
 - RM/ODP.
 - IEEE.

2000s: Líneas de Productos y Metodologías de DS basada en La Arquitectura.

Reseña histórica IV

1980

los métodos de desarrollo estructurado demostraron no escalar suficientemente y fueron dejando el lugar a un nuevo paradigma, el de la programación orientada a objetos.

Hacia fines de la década de 1980 y comienzos de la siguiente, la expresión arquitectura de software comienza a aparecer en la literatura para hacer referencia a la configuración morfológica de una aplicación.

1990

La década de 1990, creemos, será la década de la arquitectura de software. Usamos el término “arquitectura” en contraste con “diseño”, para evocar nociones de codificación, de abstracción, de estándares, de entrenamiento forma.

Como Aspecto importante podemos tomar:

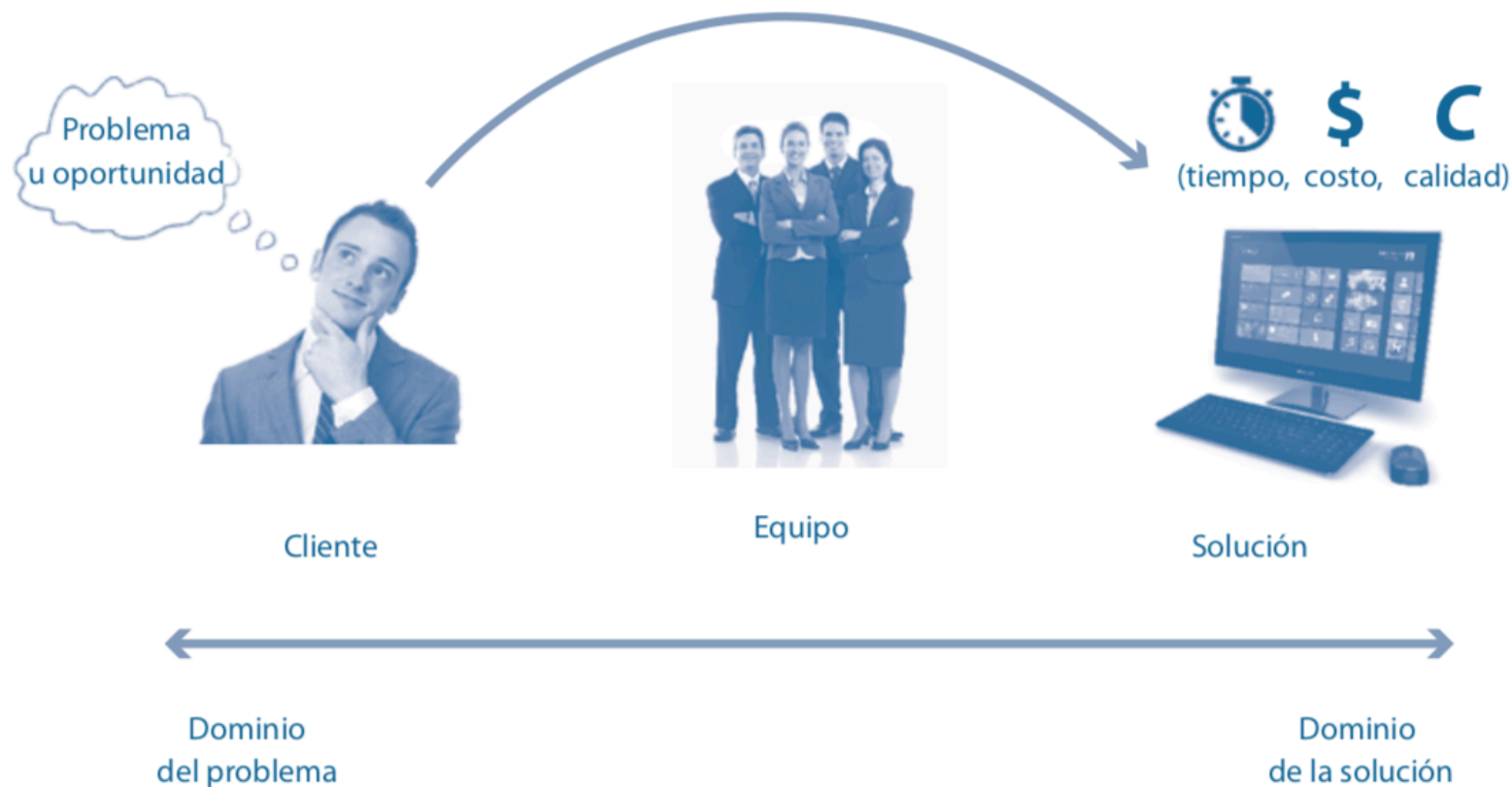
Si bien la AS acostumbra remontar sus antecedentes hasta la década de 1960, su historia no ha sido tan continua como la del campo más amplio en el que se inscribe, la ingeniería de software. Después de las tempranas inspiraciones del legendario Edsger Dijkstra, de David Parnas y de Fred Brooks, la AS quedó en estado de vida latente durante unos cuantos años, hasta comenzar su expansión explosiva con los manifiestos de Dewayne Perry de AT&T Bell Laboratories de New Jersey y Alexander Wolf de la Universidad de Colorado. Puede decirse que Perry y Wolf fundaron la disciplina, y su llamamiento fue respondido en primera instancia por los miembros de lo que podría llamarse la escuela estructuralista de Carnegie Mellon: David Garlan, Mary Shaw, Paul Clements, Robert Allen.

Reseña histórica V

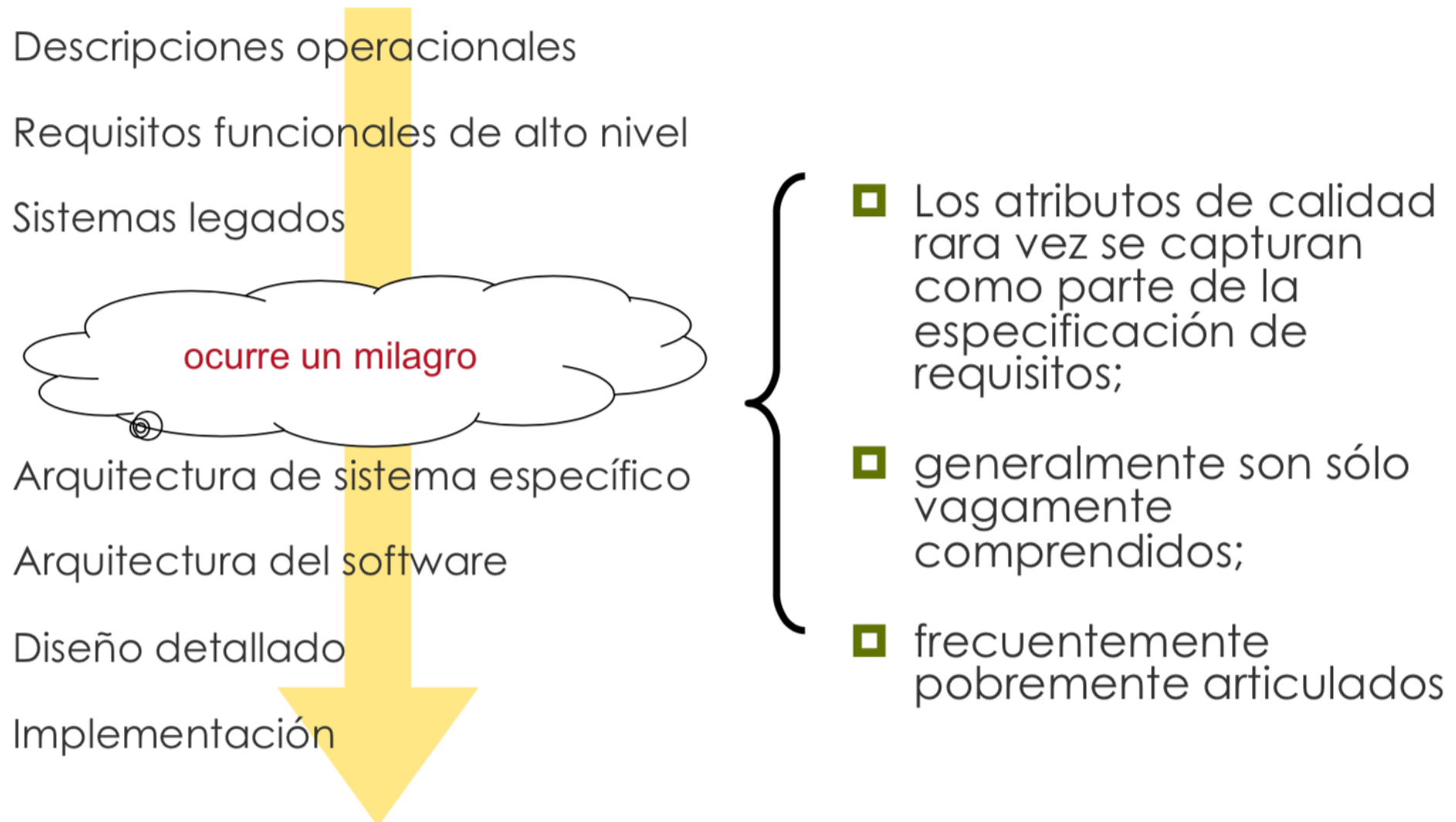
RESALTANDO QUE :

Se trata entonces de una práctica joven, de apenas unos doce años de trabajo constante, que en estos momentos experimenta una nueva ola creativa en el desarrollo cabal de sus técnicas en la obra de Rick Kazman, Mark Klein, Len Bass y otros metodólogos en el contexto del SEI, en la misma universidad. A comienzos del siglo XXI comienzan ya a discernirse tendencias, cuyas desavenencias mutuas todavía son leves: al menos una en el sur de California (Irvine y Los Ángeles) con Nenad Medvidovic, David Rosenblum y Richard Taylor, otra en el SRI de Menlo Park con Mark Moriconi y sus colegas y otra más vinculada a las recomendaciones formales de la IEEE y los trabajos de Rich Hilliard. Hoy se percibe también un conjunto de posturas europeas que enfatizan mayormente cuestiones metodológicas vinculadas con escenarios y procuran inscribir la arquitectura de software en el ciclo de vida, comenzando por la elicitación de los requerimientos. Antes de Perry y Wolf, empero, se formularon ideas que serían fundamentales para la disciplina ulterior. Comencemos entonces por el principio, aunque siempre cabrá la posibilidad de discutir cuál puede haber sido el momento preciso en el que todo comenzó.

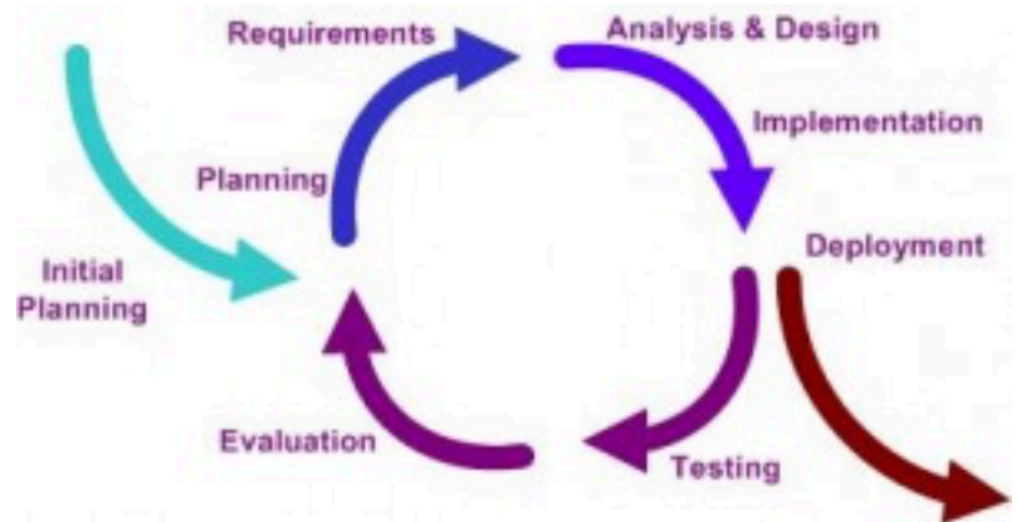
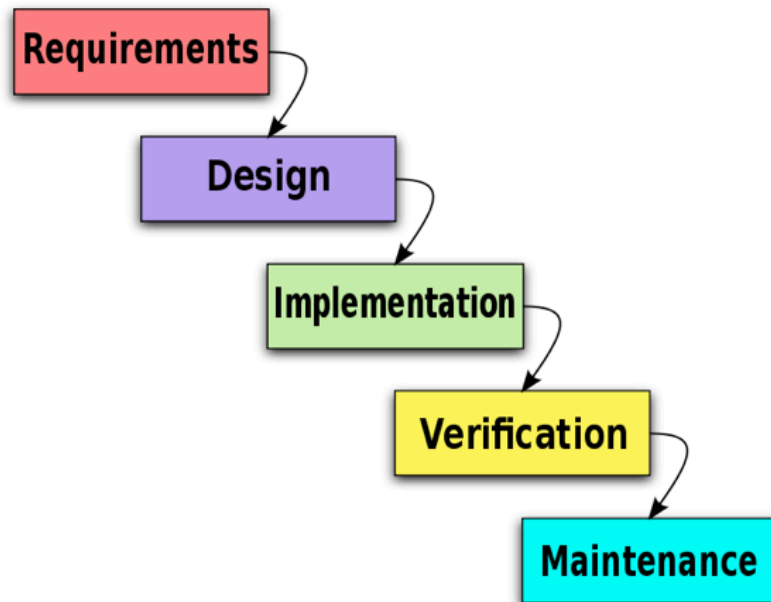
Visión simplificada del desarrollo de sistemas de software



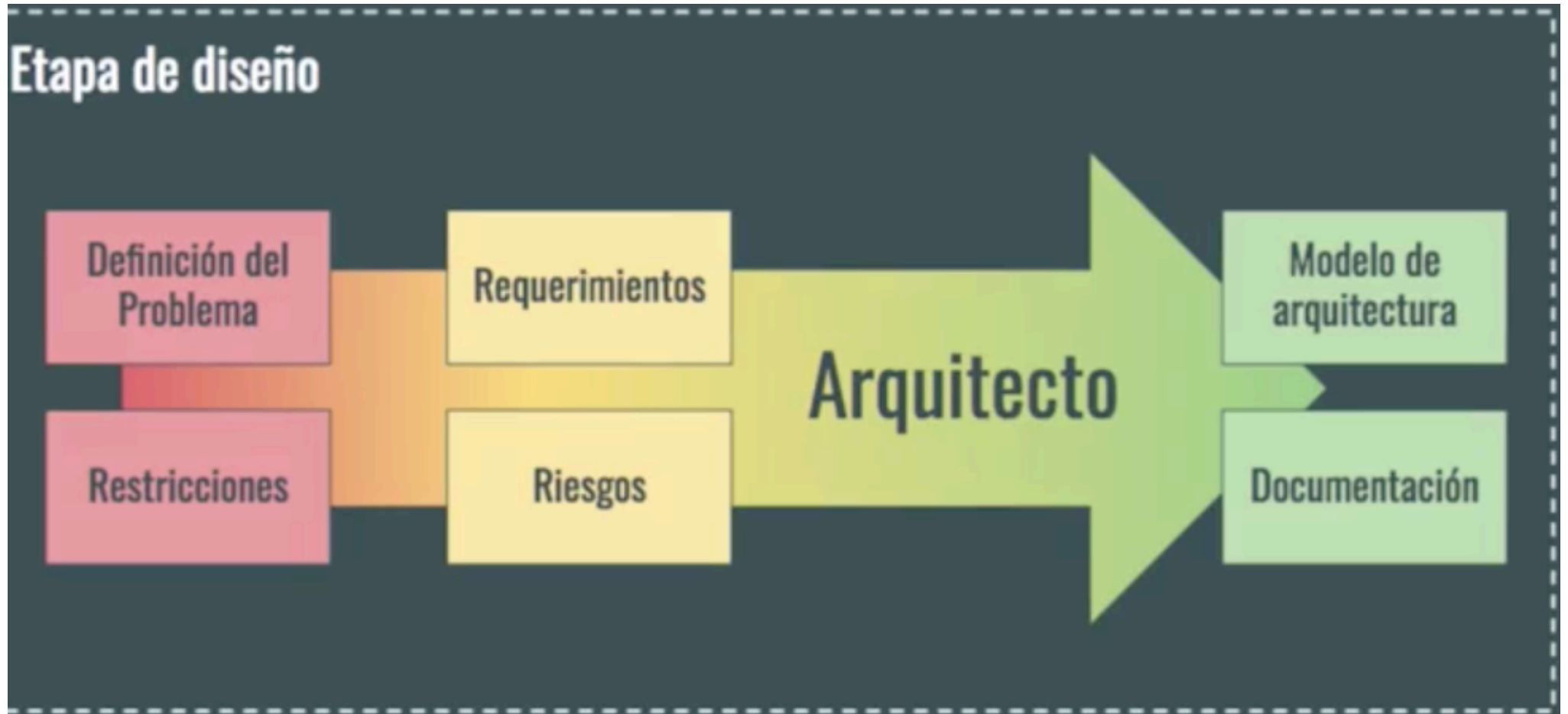
Desarrollo tradicional de SW



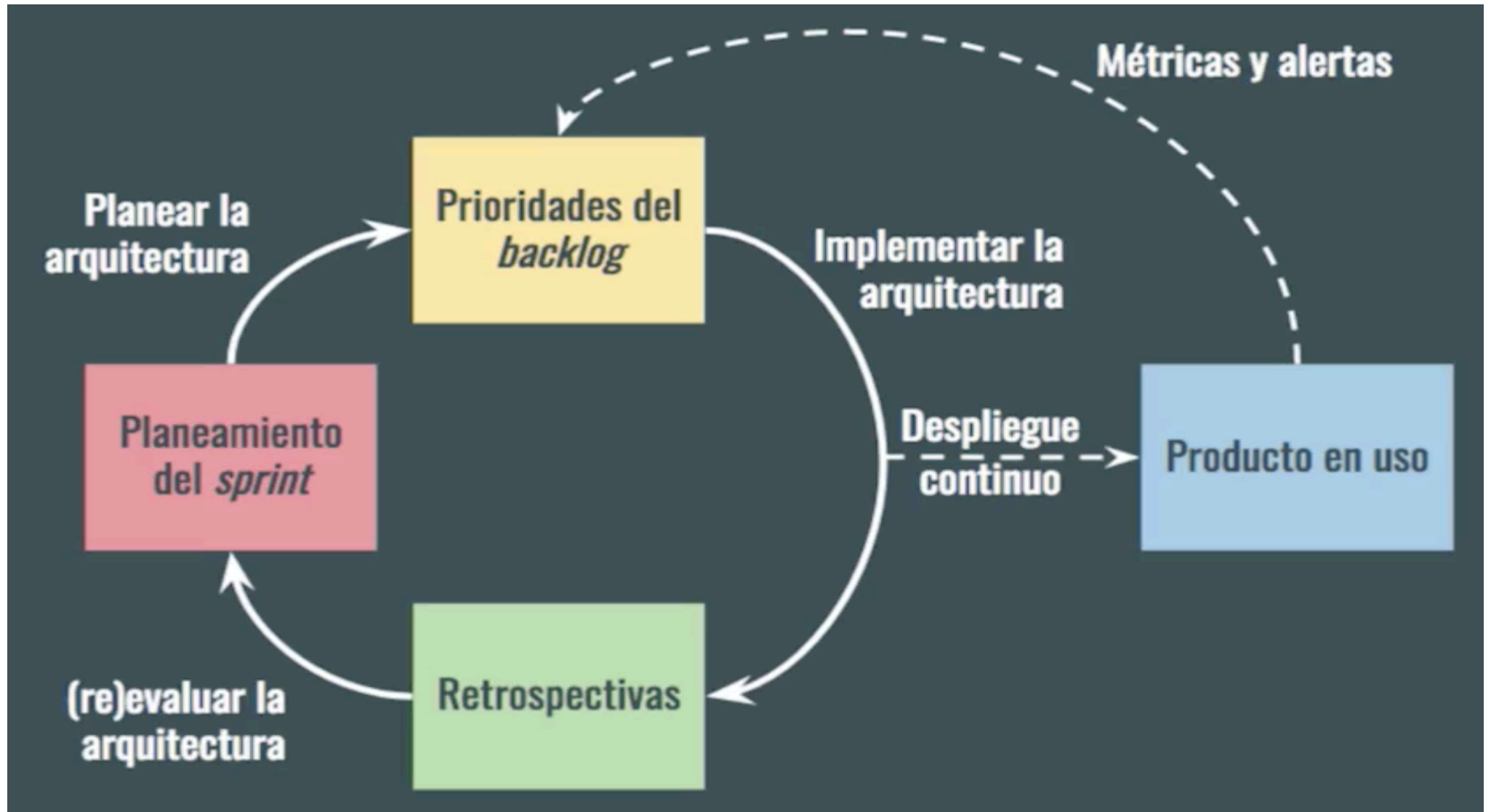
Actividades claves del Desarrollo de sistemas software



Metodologías tradicionales



Metodologías ágiles



Tracers Bullet

Esqueletos de aplicación y Prototipos

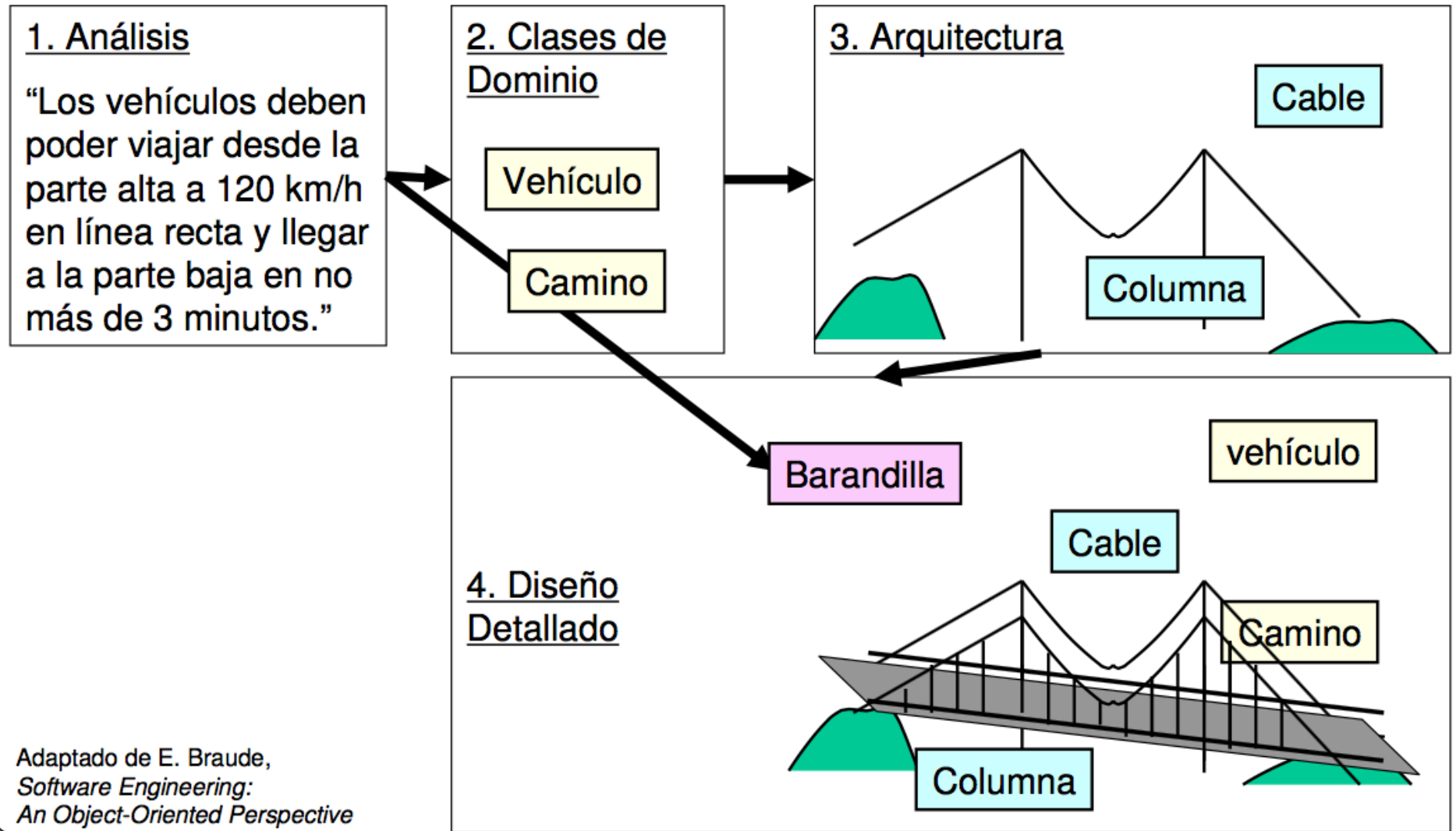


La idea de las balas trazadoras surge a partir de hacer la analogía con el uso de la artillería para dar en un objetivo. En el software análogo a disparar artillería pesada calculando todo por adelantado decimos: "Voy a especificar todo por adelantado, se lo daré a conocer a los programadores y espero que lo que salga del otro lado (producto software) esté cerca de mi objetivo (lo que el usuario necesita)".

En cambio si los requisitos (objetivo) son cambiantes, o si no conocemos todos los factores, usemos **balas trazadoras**: "Probemos y produzcamos algo realmente temprano que podamos darle al usuario para ver qué tan cerca estaremos del objetivo. A medida que pasa el tiempo, podemos ajustar nuestro objetivo ligeramente al ver dónde estamos en relación con el objetivo del usuario ". Estamos viendo pequeñas iteraciones, **código esqueleto**, que no es funcional, pero es una aplicación suficiente para mostrarle a los stakeholders cómo se mantendrá unido.

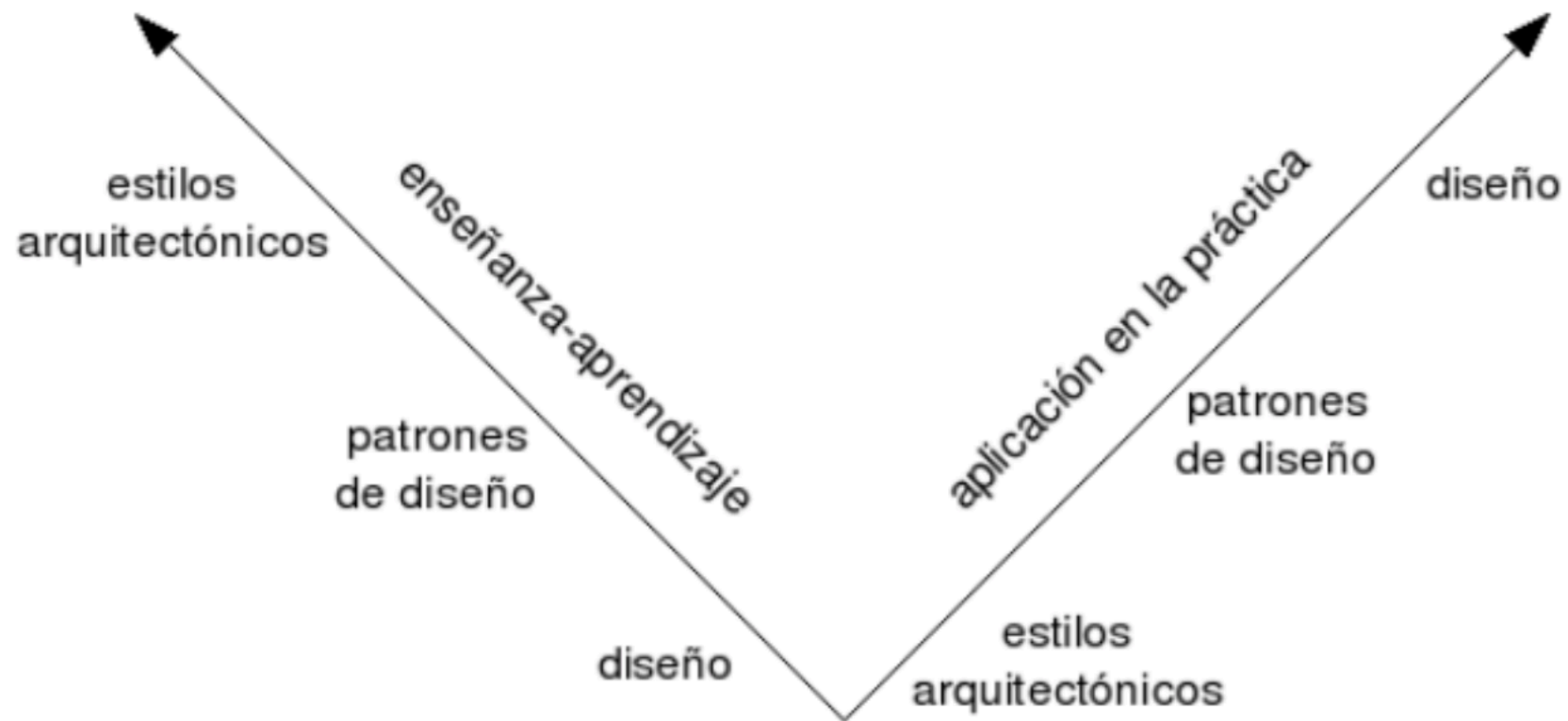
Básicamente, todo se reduce a la **retroalimentación**. Cuanto más rápido podamos obtener comentarios, menos cambios necesitaremos para volver al objetivo.

Relación entre análisis, arquitectura y diseño detallado



Adaptado de E. Braude,
*Software Engineering:
An Object-Oriented Perspective*

Diseño, Patrones y Arquitectura





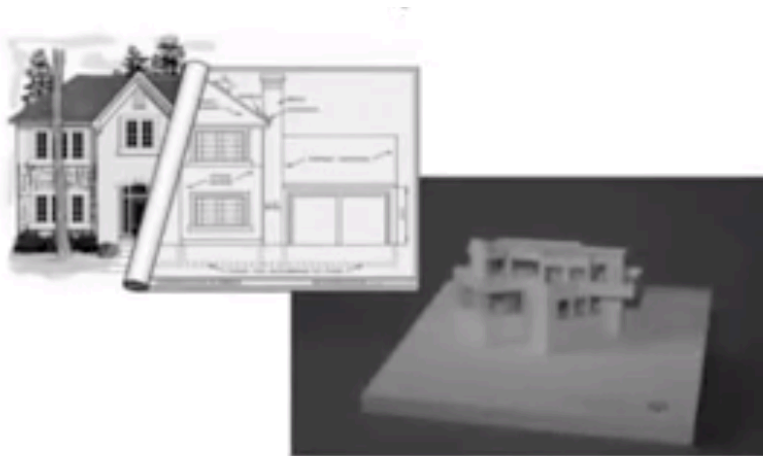
Entonces ...

¿qué es la arquitectura de software?

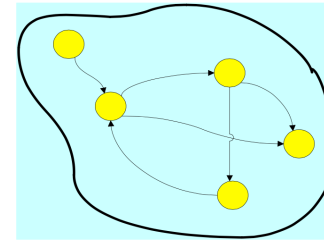
Arquitectura de Sistemas

Nos referimos a “sistemas computacionales” o “sistemas informáticos”

*“La **Arquitectura del Sistema** es la organización fundamental de un sistema incorporado en sus componentes, sus relaciones con otros y su entorno y la principales guías de su diseño y evolución” .(IEEE)*



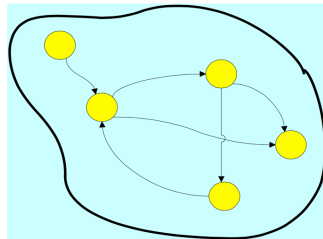
Arquitectura de Software



- Paul Clements 1996
 - La arquitectura del software es a grandes rasgos, una vista del sistema que incluye los componentes principales del mismo, la conducta de esos componentes según se percibe desde el resto del sistema y las formas en que los componentes interactúan y se coordinan para alcanzar la misión del sistema.
 - Len Bass 1998
 - La arquitectura del software de un programa o sistema de computación es la estructura o las estructuras del sistema, que contienen componentes de software, las propiedades externamente visibles de dichos componentes y las relaciones entre ellos.
- IEEE Std. 1471-2000
 - La arquitectura del software es la organización fundamental de un sistema encarnada en sus componentes, las relaciones entre ellos y con el entorno, y los principios que orientan su diseño y evolución.

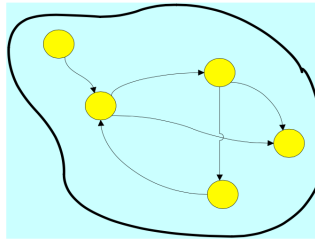
Arquitectura de Software II

La arquitectura de software de un sistema es el conjunto de estructuras necesarias para razonar sobre el sistema. Comprende elementos de software, relaciones entre ellos, y propiedades de ambos. [Bass, Clements y Kazman, 2012]



“La arquitectura de un sistema de software intensivo es la estructura o estructuras del sistema, que comprenden elementos de software, las propiedades visibles externamente de esos elementos y las relaciones entre ellos”. [Carnegie-Mellon University]

Arquitectura de Software III



“Una arquitectura es el conjunto de decisiones significativas sobre la organización de un sistema software, la selección de los elementos estructurales y sus interfaces por los que el sistema está compuesto junto con su comportamiento como se especifica en sus colaboraciones entre estos elementos, la composición de estos elementos estructurales y de comportamiento en subsistemas progresivamente más grandes y el estilo de arquitectura que guían la organización – estos elementos, sus interfaces, sus colaboraciones y su composición”. [Booch, Rumbaugh, and Jacobson]



Arquitectura de Software IV

“[...] la arquitectura se reduce a las cosas importantes, cualesquiera que sean”.

Patterns of Enterprise Application Architecture (Fowler, 2002)

Este planteo está arraigado en las metodologías ágiles, en la creencia de que la arquitectura emerge de un equipo autogestionado, por lo que no hay necesidad de hacer arquitectura por fuera el equipo de desarrollo, ya que es una actividad constante y tiene que ver con las decisiones importantes, cualesquiera que ellas sean ...



Lo esencial de la arquitectura de sw

De lo anterior podemos ver que sin importar que definición utilicemos, nos debe quedar claro que la arquitectura de software se centra en 3 elementos esenciales:

1. la estructura del sistema,
2. los componentes que lo conforman y
3. la relación que existe entre ellos.

Por lo que podemos ensayar un concepto más aproximado para definir la Arquitectura de software:

La arquitectura de software es el diseño de más alto nivel de la estructura de un sistema, el cual consiste en un conjunto de patrones y abstracciones que proporcionan un marco claro para la implementación del sistema.

Diferencias entre Arquitectura y Diseño

- El estándar ISO 12207 identifica **dos tipos de Diseño Software**:
 - **Arquitectural** [alto nivel]
 - Describe la estructura y organización de alto nivel, es decir, los subsistemas o componentes y sus relaciones
 - **Detallado**
 - Describe cada componente y su comportamiento específico, de forma que puede procederse a su construcción

Diseño arquitectónico

- Es el primer paso en el diseño de un sistema, previo al diseño detallado.
- Su resultado se conoce como **arquitectura del software**. *[se presenta después]*
- Representa el enlace entre la especificación de requisitos y el diseño.
- Puede llevarse a cabo en paralelo con actividades de especificación de requisitos.
- Implica un esfuerzo creativo, de forma que las actividades a realizar pueden cambiar según la naturaleza del sistema a desarrollar.

Importancia de la Arquitectura



- ✓ Sirve de guía
- ✓ Facilita la depuración del código
- ✓ Asegura un entendimiento común del proyecto entre las partes interesadas.
- ✓ Reduce el riesgo
- ✓ Define el límite de la aplicación software
- ✓ Disminuye costos

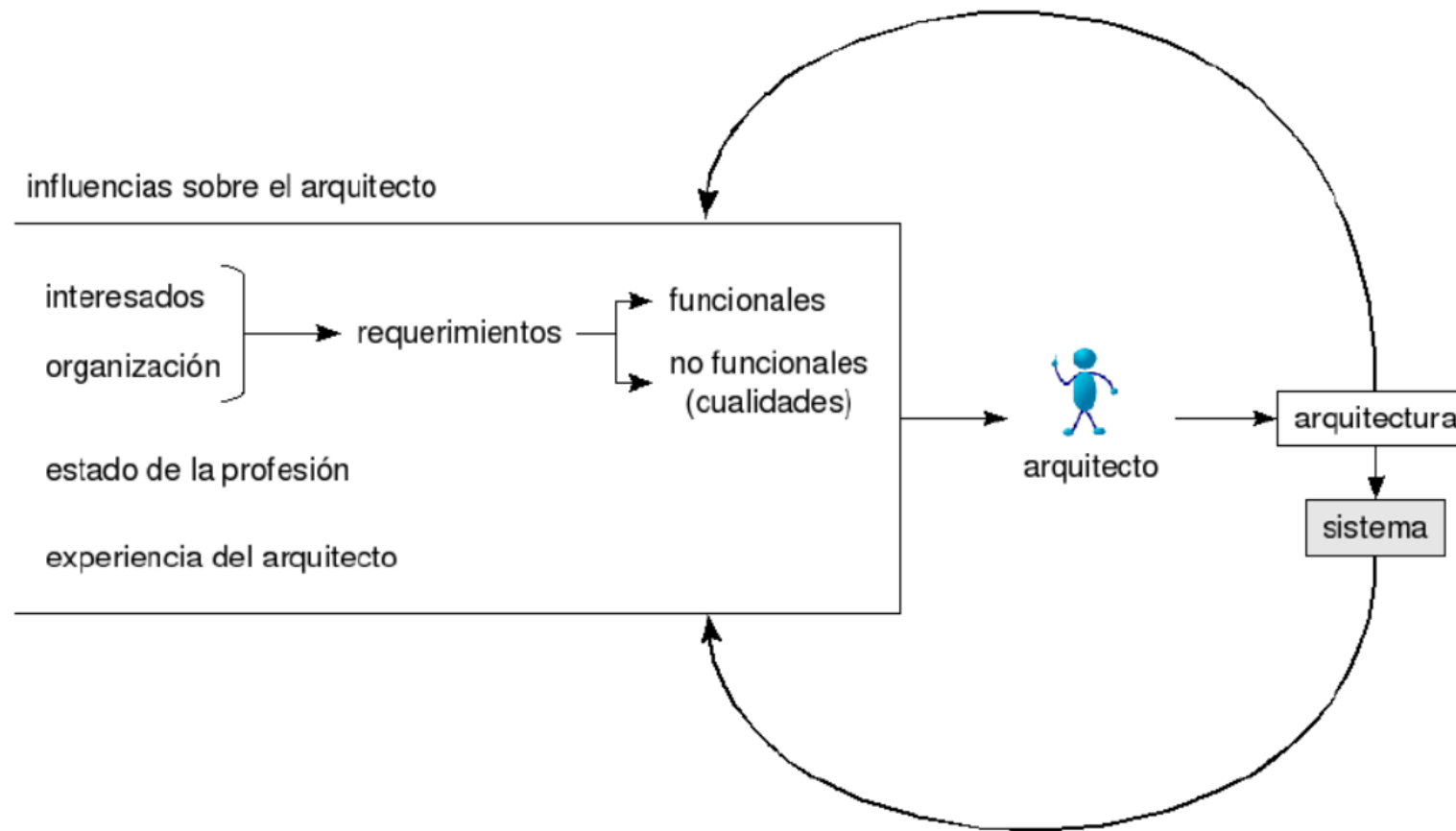


Ventajas de la Arquitectura

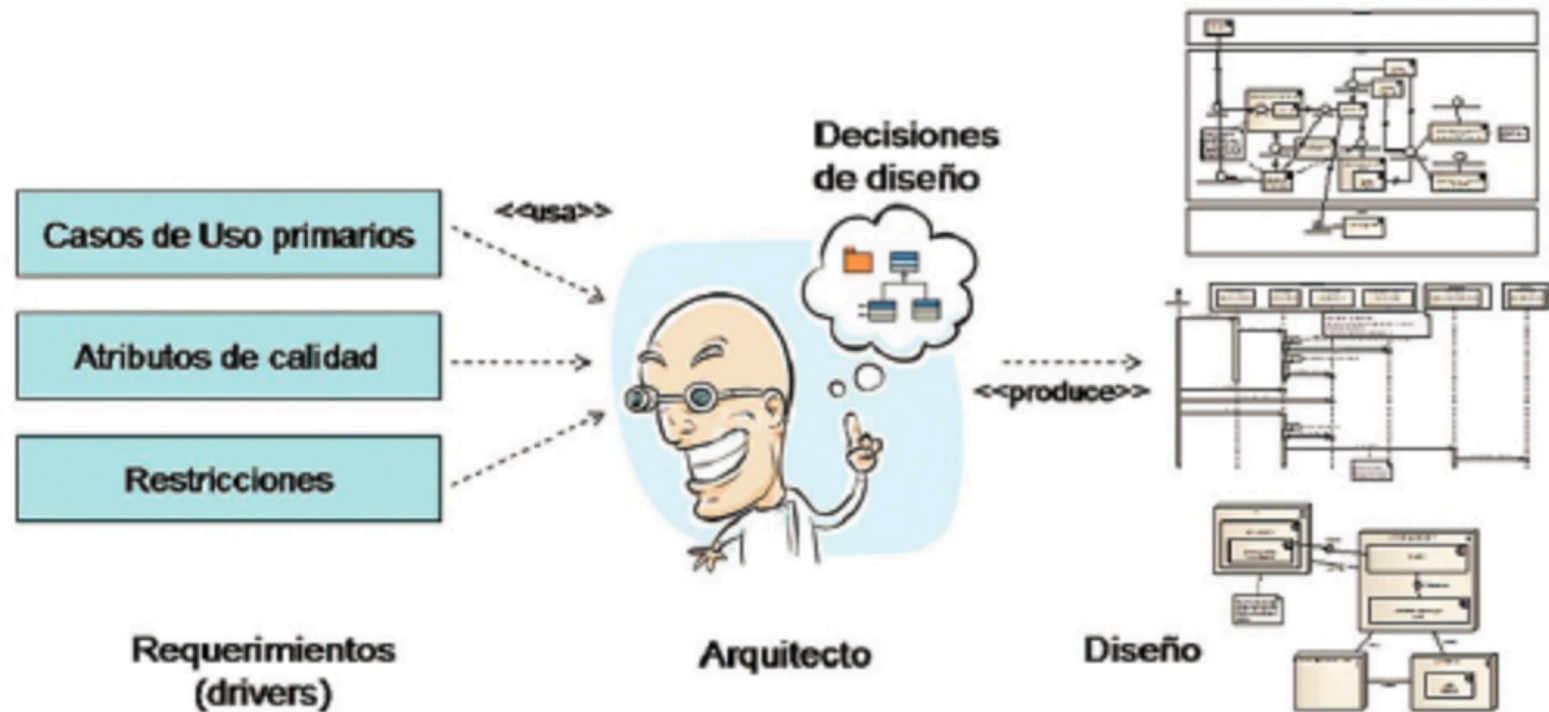
Bass y sus colaboradores (2003) analizan **tres ventajas de diseñar y documentar** de manera explícita la **arquitectura de software**:

1. **Comunicación con los participantes:** La arquitectura es una presentación de alto nivel del sistema, que puede usarse como un enfoque para la discusión de un amplio número de participantes.
2. **Análisis del sistema:** En una etapa temprana en el desarrollo del sistema, aclarar la arquitectura del sistema requiere cierto análisis. Las decisiones de diseño arquitectónico tienen un efecto profundo sobre si el sistema puede o no cubrir requerimientos críticos como rendimiento, fiabilidad y mantenibilidad.
3. **Reutilización a gran escala:** Un modelo de una arquitectura de sistema es una descripción corta y manejable de cómo se organiza un sistema y cómo interoperan sus componentes. Por lo general, la arquitectura del sistema es la misma para sistemas con requerimientos similares y, por lo tanto, puede soportar reutilización de software a gran escala.

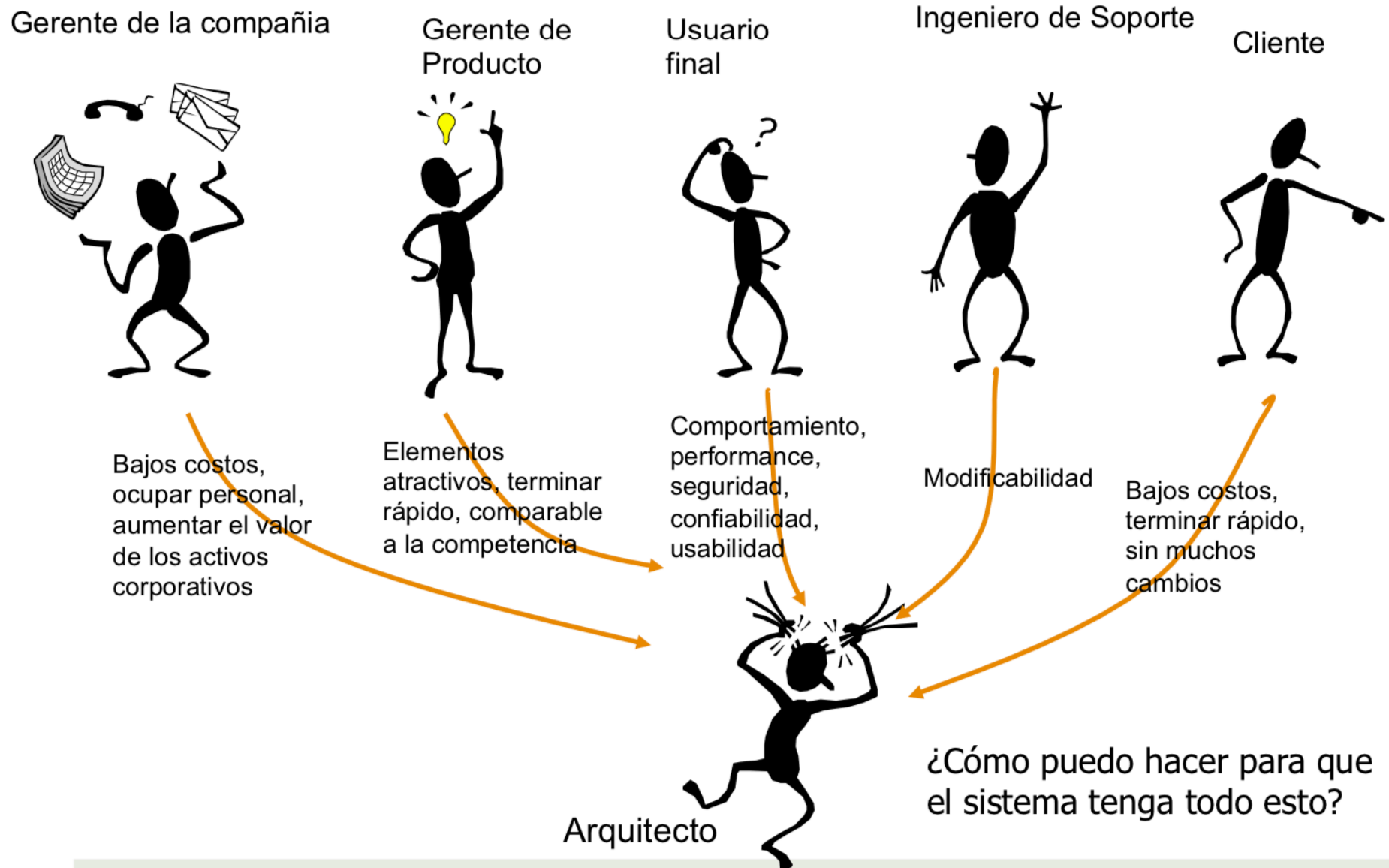
El rol del arquitecto de SW



El rol del arquitecto de SW



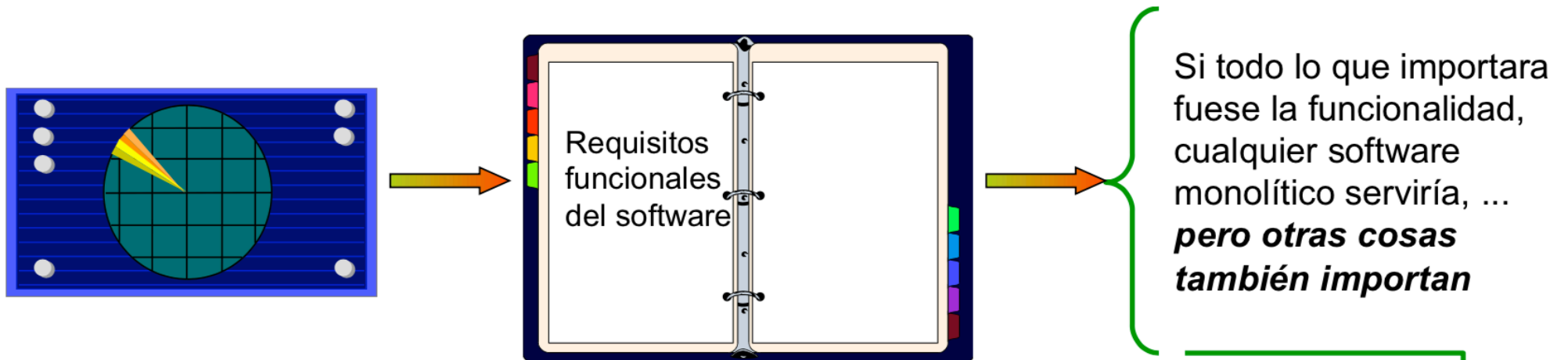
Influencia de los stakeholders



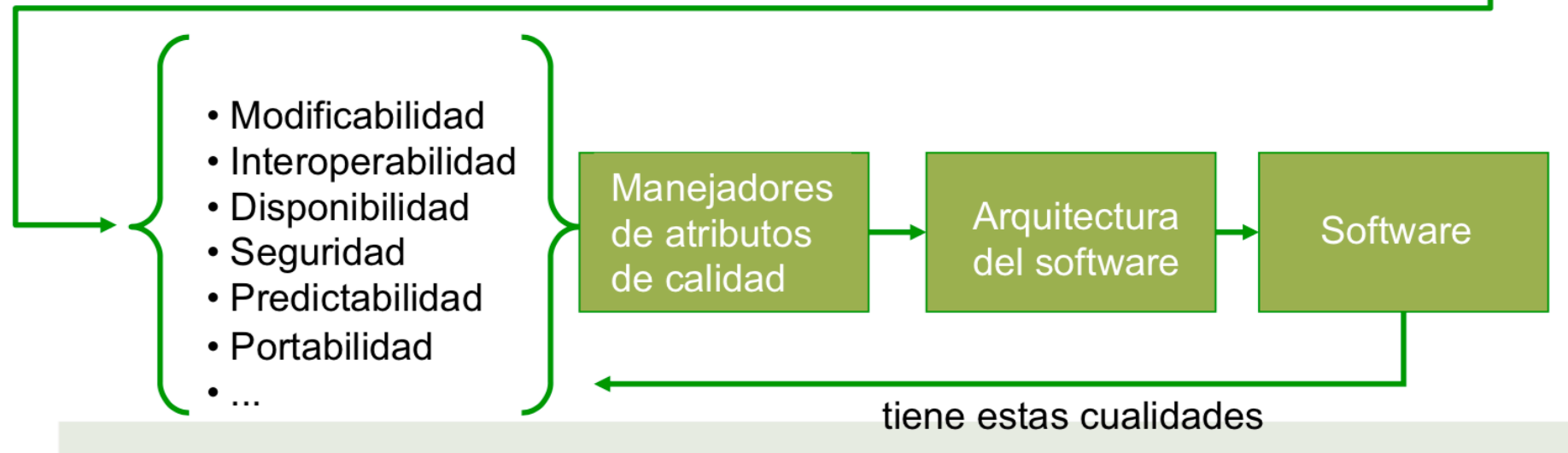
Interesados involucrados

- Los objetivos organizacionales y las propiedades del sistema requeridas por el negocio raramente se comprenden y menos aún se articulan completamente.
- Los requisitos de calidad del cliente casi nunca se documentan, lo cual resulta en:
 - objetivos que no se alcanzan;
 - conflicto inevitable entre los interesados.
- Los arquitectos deben identificar e involucrar activamente a los interesados de modo de:
 - comprender las restricciones reales del sistema;
 - administrar las expectativas de los interesados;
 - negociar las prioridades del sistema;
 - tomar decisiones de compromiso.

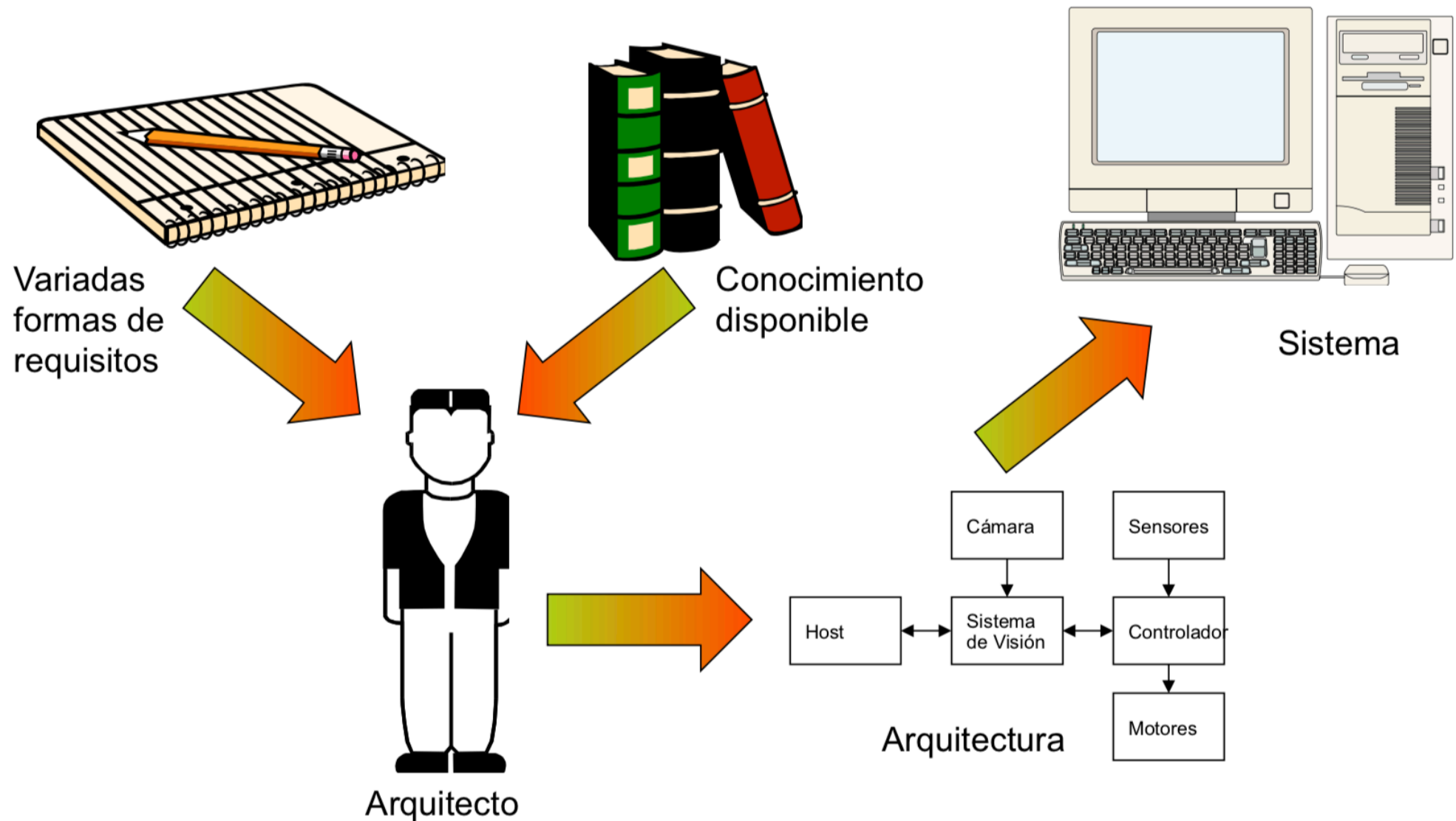
Déjà vu: desarrollo de SW



Los atributos de calidad del software y su caracterización son esenciales.



Los requisitos determinan el modelo



Decisiones de arquitectura y atributos de calidad



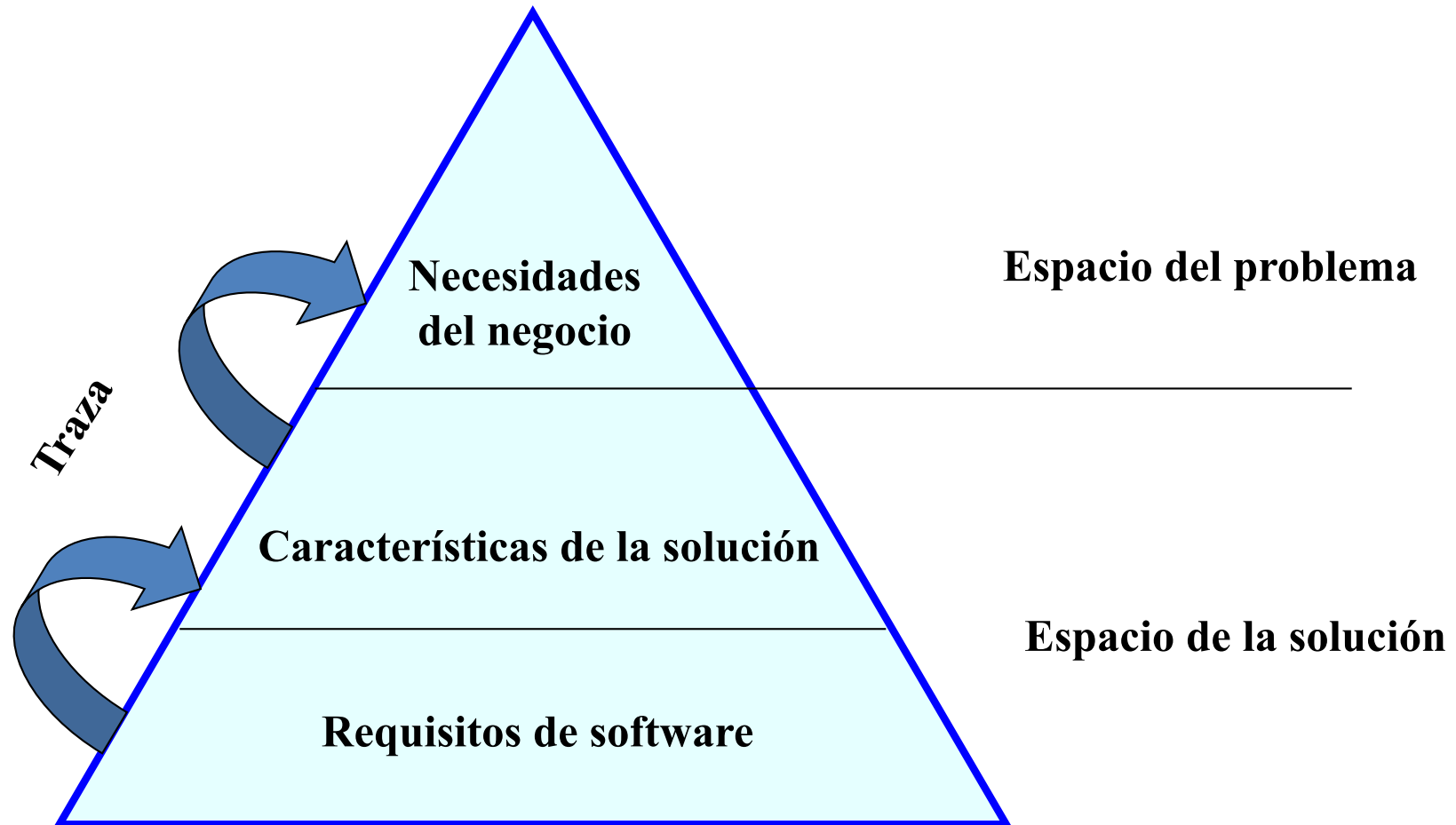
- La medida en que un sistema alcanza sus requisitos de calidad depende de las decisiones de arquitectura:
 - la arquitectura es crítica para alcanzar los atributos de calidad;
 - las cualidades del producto deben diseñarse como parte de la arquitectura;
 - un cambio en la estructura que mejora una cualidad suele afectar las otras cualidades;
 - la arquitectura sólo puede permitir, no garantizar, que cualquier requisito de calidad se alcance.



En busca de los requisitos significativos para la Arquitectura

(refrescando lo aprendido)

Trazabilidad de requisitos



ESPACIO DEL PROBLEMA

V S

ESPACIO DE SOLUCIÓN

A LA HORA DE TOMAR REQUERIMIENTOS Y ANALIZARLOS ES MUY IMPORTANTE ENTENDER EL PROBLEMA QUE ESTAMOS RESOLVIENDO.

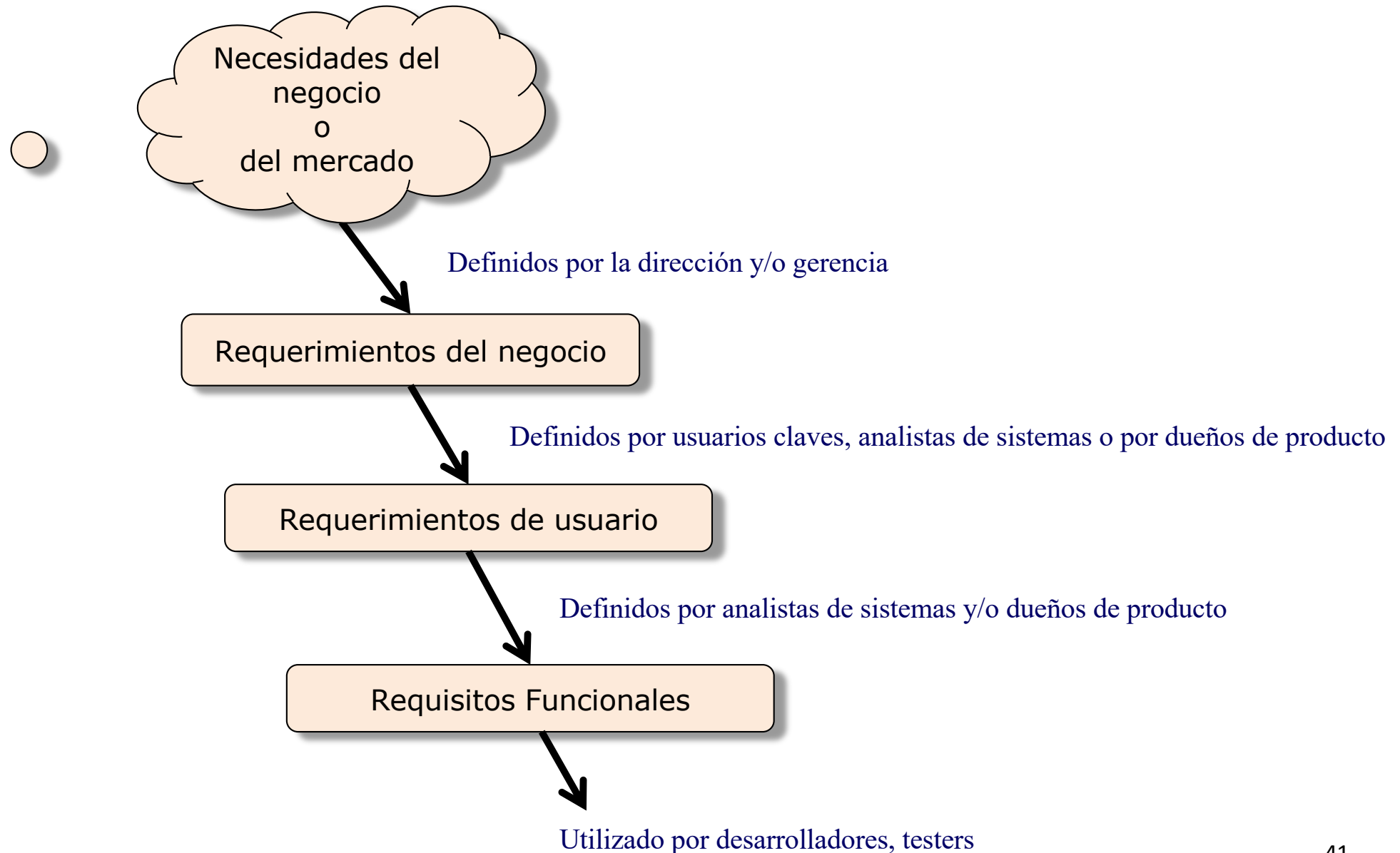


Entender el problema

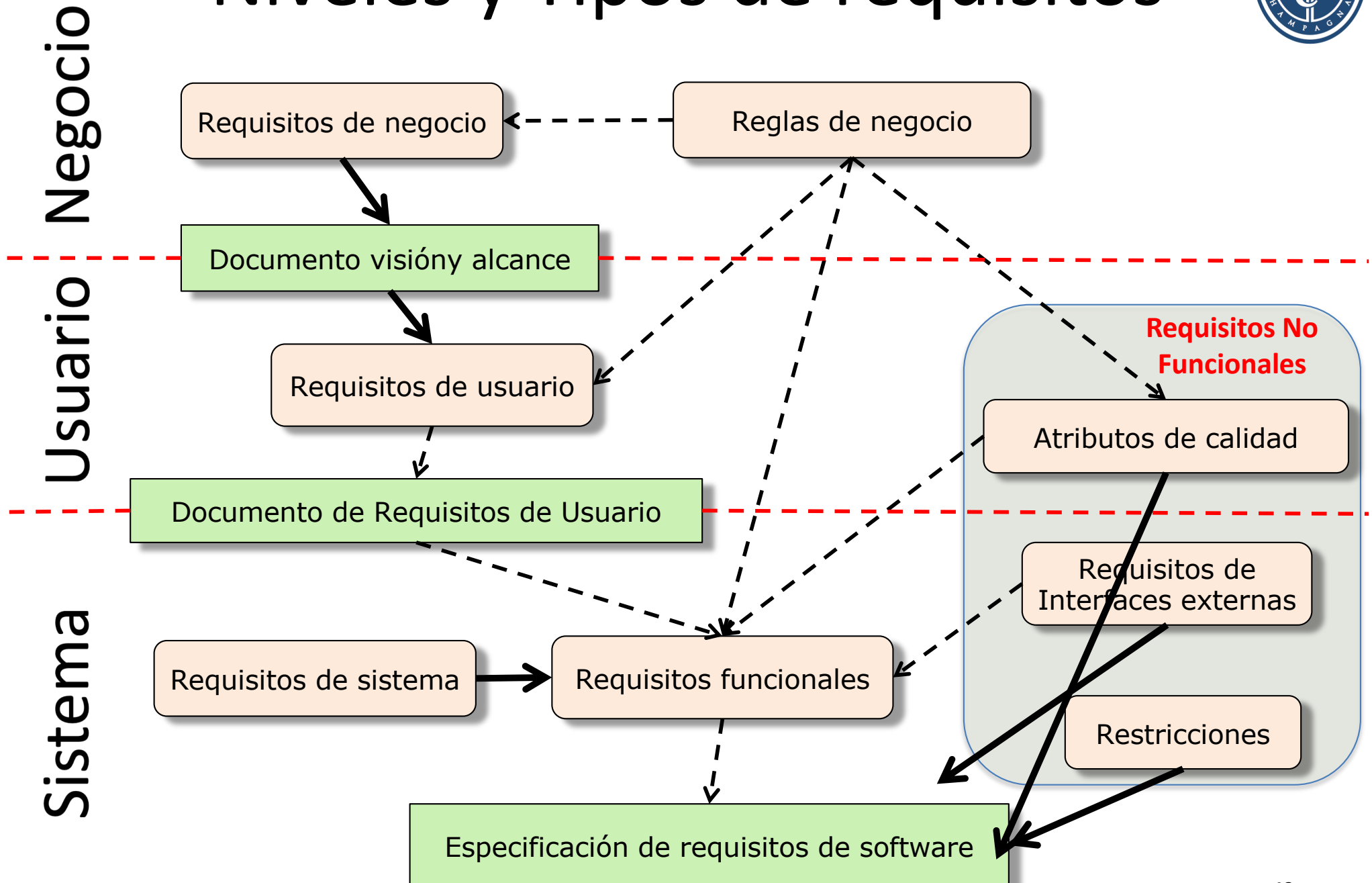
Entender el problema: Estudiar el dominio del problema es una de las actividades necesarias para identificar las necesidades de negocio de clientes y usuarios dentro de la Ingeniería de Requisitos.

El objetivo principal del “estudio del problema” es obtener un conocimiento suficiente como para poder comunicarse eficazmente con clientes y usuarios, comprender su negocio, entender sus necesidades y poder proponer una solución adecuada.

¿Cómo logramos entender el problema?



Niveles y Tipos de requisitos



Tipos de Requisitos / Requerimientos



Requerimientos funcionales

“Como personal de enfermería quiero ver la información del estado del paciente para poder reaccionar a cualquier anomalía.”



Un requisito funcional es una descripción del comportamiento esperado de un sistema, la cual es por lo general solicitada de forma explícita.

Requerimientos no funcionales

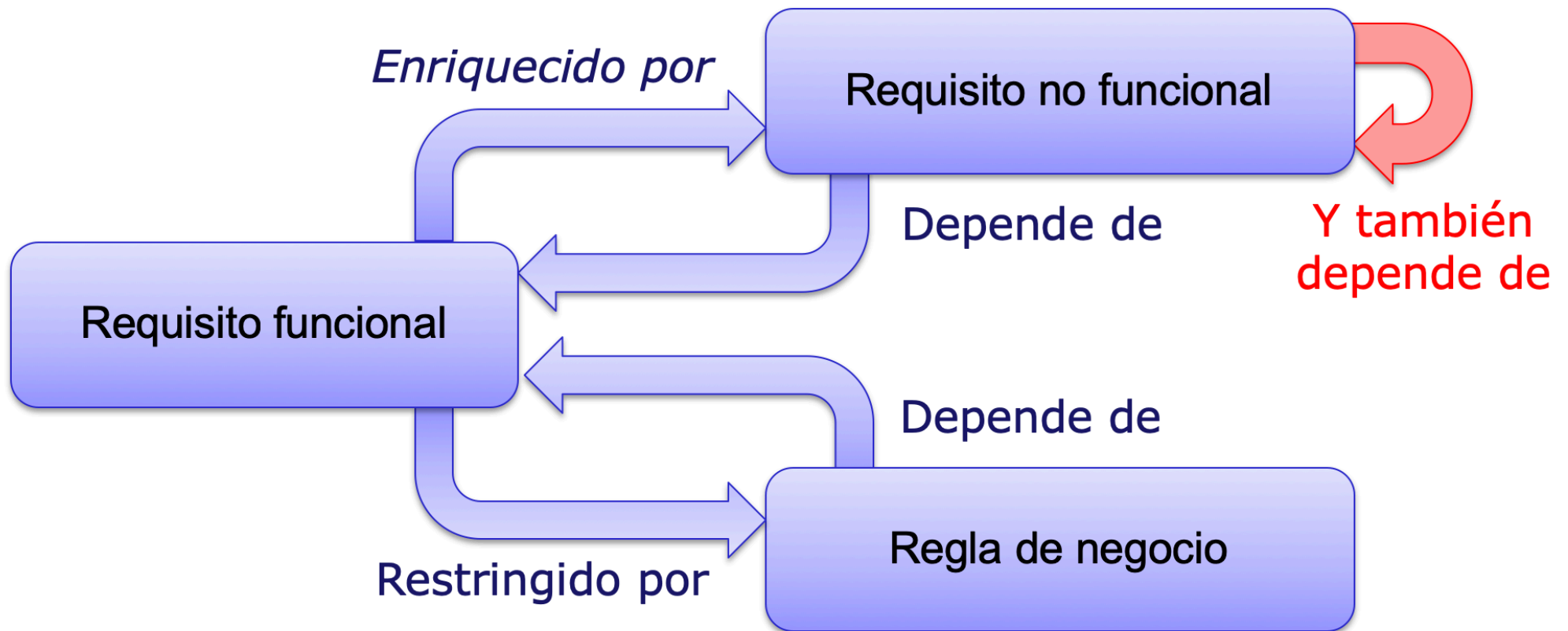
“Como personal de enfermería quiero ver la información en tiempo real del estado del paciente para poder reaccionar a cualquier anomalía.”



Un requisito NO funcional es una descripción de una propiedad o característica de un sistema, la cual es rara vez solicitada de forma explícita.

Dependencias entre tipos de requisitos

Los requisitos deben verse como una unidad compuesta de tres partes mutuamente necesarias:





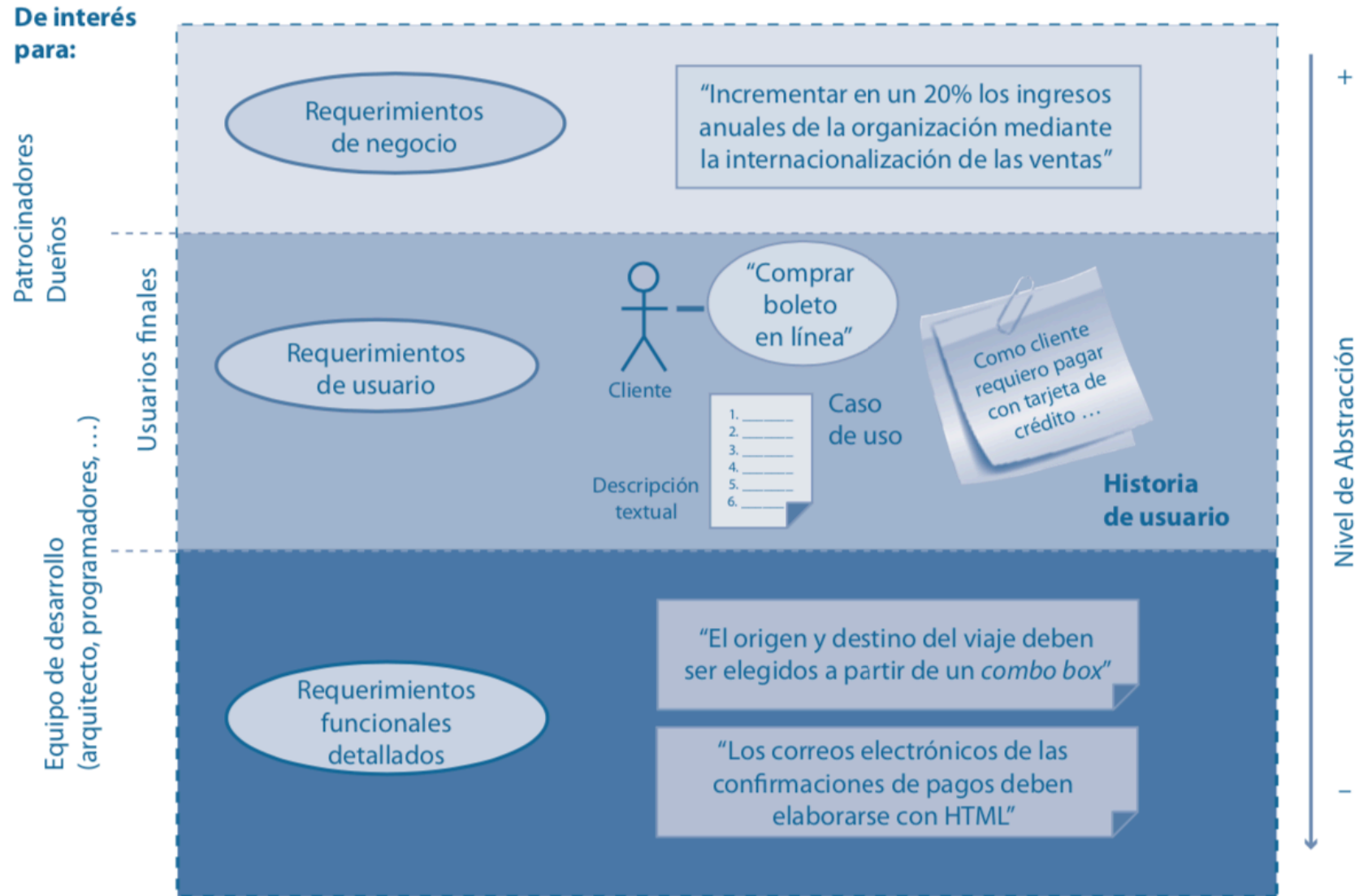
¿Requisitos de arquitectura?

La priorización de los requisitos que tienen mayor influencia en la forma que tomarán los elementos que componen las estructuras de la arquitectura, es clave en el desarrollo de software. Este subconjunto de requisitos se los denomina **requisitos significativos para la arquitectura o como los denominan algunos autores “drivers arquitectónicos”**.

En el contexto del proceso de desarrollo de la arquitectura, la etapa de requisitos se centra en estos “drivers” y los podemos clasificar en función de sus fuentes:

1. *Drivers funcionales.*
2. *Drivers de atributos de calidad.*
3. *Drivers de restricciones.*

Requisitos funcionales



Arquitectura y funcionalidad

- La funcionalidad es en gran medida ortogonal a los requisitos de calidad:
 - La funcionalidad es la capacidad del sistema de hacer lo que se pretendía que hiciese;
 - Los sistemas se descomponen en elementos para lograr variados propósitos, más allá de la funcionalidad:
 - Las opciones de arquitectura promueven ciertas cualidades al tiempo que implementan la funcionalidad deseada.

Atributos de Calidad - AC





Atributos de Calidad II

1. **Disponibilidad.** Indicador acerca de si el sistema se encuentra en una condición operable cuando requiere ser utilizado.
2. **Seguridad.** Indicador del grado de protección ante usos o accesos inapropiados del sistema.
3. **Desempeño.** Indicador sobre la cantidad de trabajo realizado por el sistema considerando tiempo y recursos.
4. **Facilidad de prueba.** Indicador acerca de la facilidad con la cual se elaboran pruebas efectivas para el sistema.
5. **Modificabilidad.** Indicador referente al costo de realizar cambios en el sistema.
6. **Usabilidad.** Indicador sobre la facilidad con la cual el sistema puede ser utilizado por los usuarios.
10. **Interoperabilidad.** Indicador acerca de la facilidad del sistema para intercambiar información con otros sistemas mediante interfaces.

Especificación de Atributos de Calidad



Requisito	Especificación
Seguridad, interoperabilidad, robustez, confiabilidad, usabilidad, precisión	En los requisitos funcionales
Eficiencia, performance, flexibilidad, confiabilidad	En la arquitectura del sistema
Interoperabilidad, usabilidad	Como restricciones de diseño
Flexibilidad, mantenibilidad, portabilidad, confiabilidad, reusabilidad, verificabilidad, usabilidad	Como guías de diseño
Portabilidad	Como restricciones de implementación



AC y estilos arquitectónicos

Debido a la estrecha relación entre los requerimientos no funcionales y la arquitectura de software, el estilo y la estructura arquitectónicos particulares que se elijan para un sistema dependerán de los requerimientos de sistema no funcionales:

- 1. Rendimiento:** Si el rendimiento es un requerimiento crítico, la arquitectura debe diseñarse para localizar operaciones críticas dentro de un pequeño número de componentes, con todos estos componentes desplegados en la misma computadora en vez de distribuirlos por la red. Esto significaría usar algunos componentes relativamente grandes, en vez de pequeños componentes de grano fino, lo cual reduce el número de comunicaciones entre componentes. También puede considerar organizaciones del sistema en tiempo de operación que permitan a éste ser replicable y ejecutable en diferentes procesadores.
- 2. Seguridad:** Si la seguridad es un requerimiento crítico, será necesario usar una estructura en capas para la arquitectura, con los activos más críticos protegidos en las capas más internas, y con un alto nivel de validación de seguridad aplicado a dichas capas.



AC y estilos arquitectónicos II

2. **Protección:** Si la protección es un requerimiento crítico, la arquitectura debe diseñarse de modo que las operaciones relacionadas con la protección se ubiquen en algún componente individual o en un pequeño número de componentes. Esto reduce los costos y problemas de validación de la protección, y hace posible ofrecer sistemas de protección relacionados que, en caso de falla, desactiven con seguridad el sistema.
3. **Disponibilidad:** Si la disponibilidad es un requerimiento crítico, la arquitectura tiene que diseñarse para incluir componentes redundantes de manera que sea posible sustituir y actualizar componentes sin detener el sistema.
4. **Mantenibilidad:** Si la mantenibilidad es un requerimiento crítico, la arquitectura de sistema debe diseñarse usando componentes autocontenidos de grano fino que pueden cambiarse con facilidad. Los productores de datos tienen que separarse de los consumidores y hay que evitar compartir las estructuras de datos.

Arquitectura: conflictos

- Rendimiento vs Mantenibilidad
 - Usar componentes grandes mejora el rendimiento pero reduce la mantenibilidad.
- Disponibilidad vs Seguridad
 - Introducir datos redundantes mejora la disponibilidad pero hace más difícil la seguridad.
- Protección vs Rendimiento
 - Localizar las características de protección en diversos componentes suele significar más comunicación y por tanto, un rendimiento peor.

Relaciones de contribución entre atributos de calidad



Sentido de lectura →

Atributos actuales en la aplicación

Atributos a modificar en la aplicación

	Availability	Efficiency	Installability	Integrity	Interoperability	Modifiability	Performance	Portability	Reliability	Reusability	Robustness	Safety	Scalability	Security	Usability	Verifiability
Availability								+	+							
Efficiency	+			-	-	+	-		-		+			-		
Installability	+							+					+			
Integrity		-		-					-		+		+	-	-	
Interoperability	+	-	-				+	+	+		-		-			
Modifiability	+	-						+	+			+			+	
Performance		+		-	-					-		-		-		
Portability		-		+	-	-			+				-	-	+	
Reliability	+	-		+	+	-				+	+		+	+	+	
Reusability		-		+	+	-	+						-		+	
Robustness	+	-	+	+	+			+			+	+	+	+		
Safety		-		+	+					+			+	-	-	
Scalability	+	+		+		+	+	+	+							
Security	+			+	+	-	+	+	+	+	+			-	-	
Usability		-	+				+	+	+	+	+					
Verifiability	+		+	+	+			+	+	+	+		+	+		

Relaciones de contribución entre atributos de calidad - II

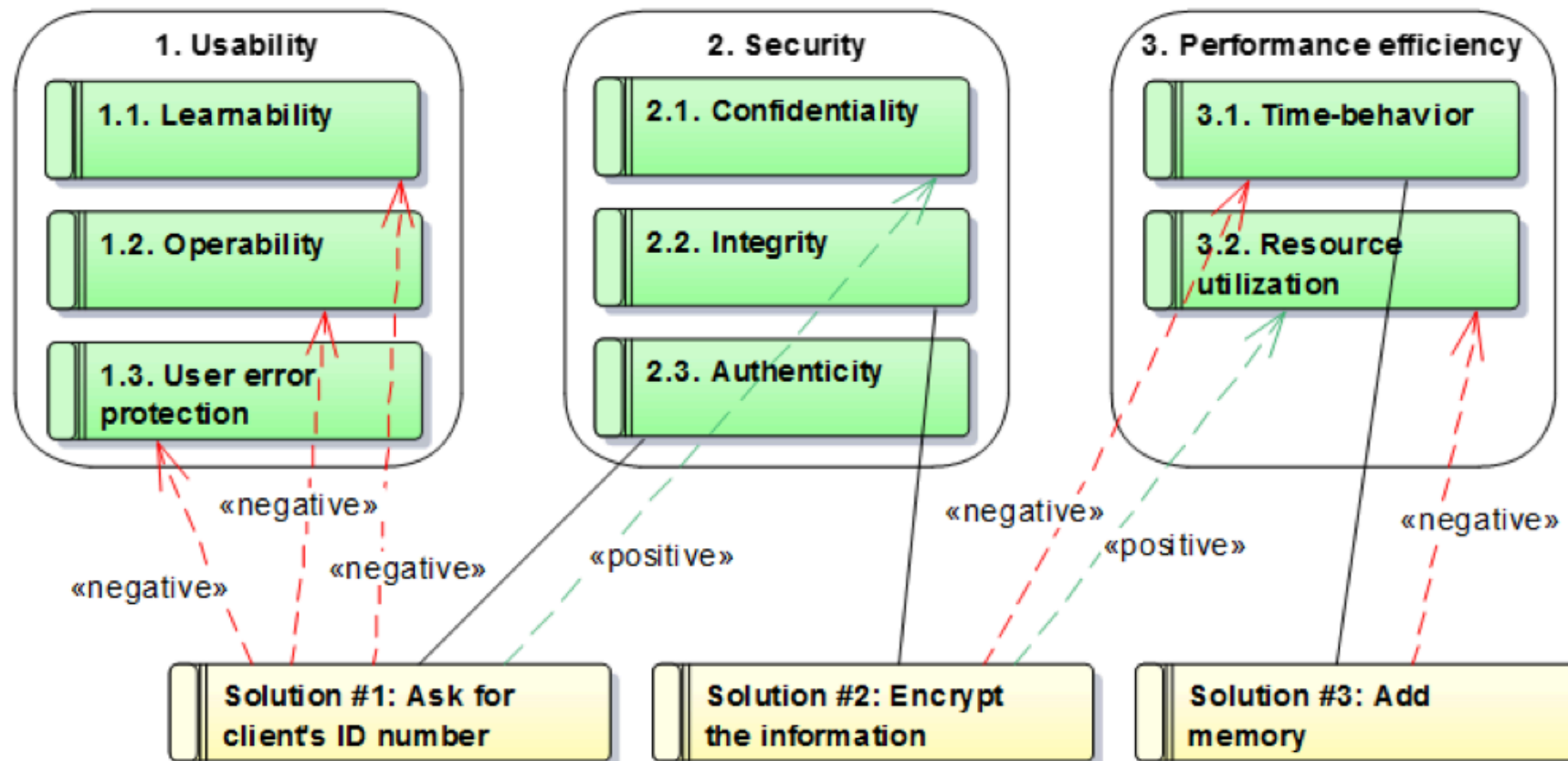
Atributos actuales en la aplicación

Sentido de lectura →

Atributos a modificar en la aplicación

	Availability	Efficiency	Installability	Integrity	Interoperability	Modifiability	Performance	Portability	Reliability	Reusability	Robustness	Safety	Scalability	Security	Usability	Verifiability	Req. funcional
Availability									+	+							
Efficiency	+			-	-	+	-			-		+			-		
Installability	+							+						+			
Integrity			-		-				-		+		+		-	-	
Interoperability	+		-	-			+	+		+	-						
Modifiability	+								+	+							
Performance		+		-	-					-							
Portability		-		+	-	-			+						-	+	
Reliability	+	-		+	+	-				+	+		+	+	+		
Reusability		-		+	+	-	+							-		+	
Robustness	+	-	+	+	+			+				+	+	+	+		
Safety		-		+	+					+				+	-	-	
Scalability	+	+		+		+	+	+		+							
Security	+			+	+		-	-	+	+	+				-	-	
Usability		-	+				-	-	+	+	+					-	
Verifiability	+		+	+		+			+	+	+	+	+	+	+		
Req. funcional																	

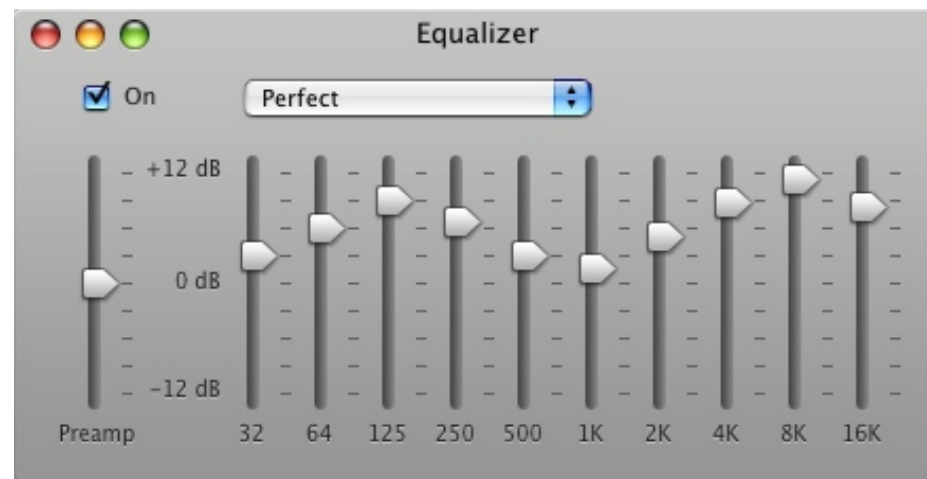
Administrando el impacto de la solución



Administrando el impacto de la solución - II



- Cuando diseñamos una solución para mejorar un atributo de calidad, necesariamente nos enfrentaremos a efectos indeseados a causa de las relaciones de contribución positivas y negativas entre los atributos de calidad.
- Estos efectos pueden ser realmente importantes, y debemos controlarlos.





Restricciones

las restricciones describen aspectos que limitan el proceso de desarrollo del sistema.

Para facilitar su manejo se distinguen dos subclases:

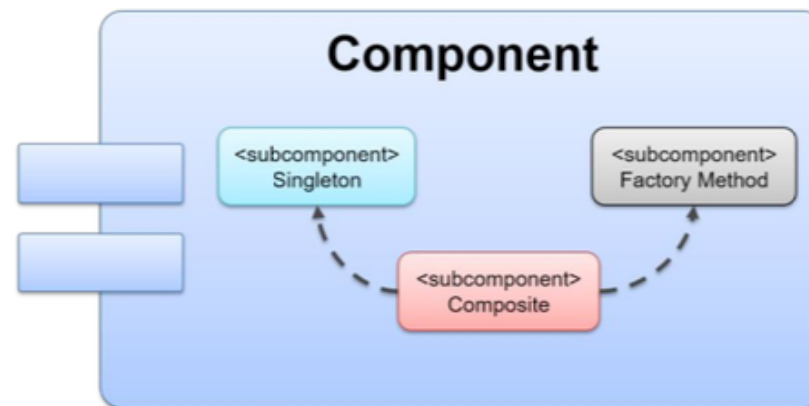
- 1. Restricciones técnicas:** a menudo son solicitudes expresas sobre el uso, durante el desarrollo del sistema, de productos de *software* provistos por terceros, métodos de diseño o implementación, productos de *hardware* o lenguajes de programación. Ej.: gestor de BD freeware.
- 2. Restricciones administrativas:** describen aspectos que restringen el proceso de desarrollo del sistema. Ej.: costo y tiempo de desarrollo.



Algunos concepto claves

Patrones de diseño

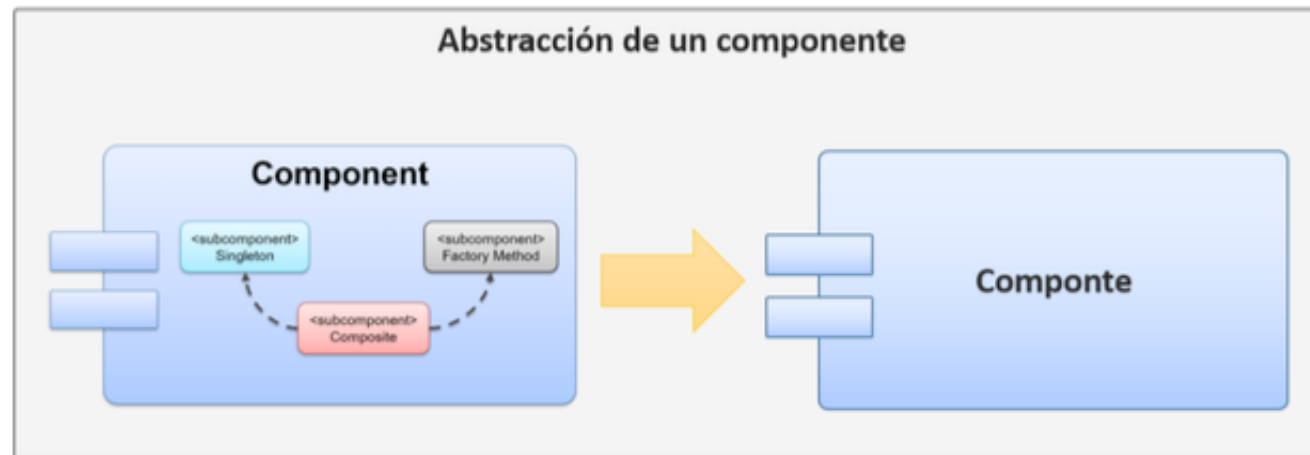
Si recordamos lo aprendido sobre patrones de diseño podemos decir, como regla general, que los patrones de diseño se centran en cómo las clases y los objetos se crean, relacionan, estructuran y se comportan en tiempo de ejecución, pero siempre, centrado en las clases y objetos, y nunca en componentes.



Podríamos decir que ven al componente bajo el concepto de caja blanca (se enfoca en su interior).

Abstracción de un componente

Cuando un arquitecto de software analiza un componente, siempre deberá crear abstracciones para facilitar su análisis, de tal forma, que eliminará todos aquellos aspectos que no son relevantes para la arquitectura y creará una representación lo más simple posible.



“Abstracción de un componente”: Es el proceso por medio del cual aislamos a un elemento de su contexto o resto de elementos que lo acompañan con el propósito de crear una representación más simple de él, el cual se centra en lo que hace y no en como lo hace.

Patrones arquitectónicos

- “Un **patrón arquitectónico** es una descripción de una organización del sistema” (Garlan y Shaw, 1993)
- “Expresa una organización estructural fundamental o esquema para sistemas de software. Proporciona un conjunto de subsistemas predefinidos, especifica sus responsabilidades e incluye reglas y pautas para organizar las relaciones entre ellos.” **TOGAF**
- “Los patrones arquitectónicos son un método para organizar bloques de funcionalidad para satisfacer una necesidad. Los patrones se pueden utilizar a nivel de software, sistema o empresa. Los patrones bien expresados le dicen cómo usarlos, y cuándo. Los patrones se pueden caracterizar según el tipo de solución a la que se dirigen.” - **MITRE**
- “Los patrones de una arquitectura expresan una organización estructural fundamental o esquema para sistemas complejos. Proporciona un conjunto de subsistemas predefinidos, especifica sus responsabilidades únicas e incluye las reglas y pautas que permiten la toma de decisiones para organizar las relaciones entre ellos. El patrón de arquitectura para un sistema de software ilustra la estructura de nivel macro para toda la solución de software. Un patrón arquitectónico es un conjunto de principios y un patrón de grano grueso que proporciona un marco abstracto para una familia de sistemas. Un patrón arquitectónico mejora la partición y promueve la reutilización del diseño al proporcionar soluciones a problemas recurrentes con frecuencia. Precisamente hablando, un patrón arquitectónico comprende un conjunto de principios que dan forma a una aplicación.” - **Architectural Patterns - Pethuru Raj, Anupama Raman, Harihara Subramanian**

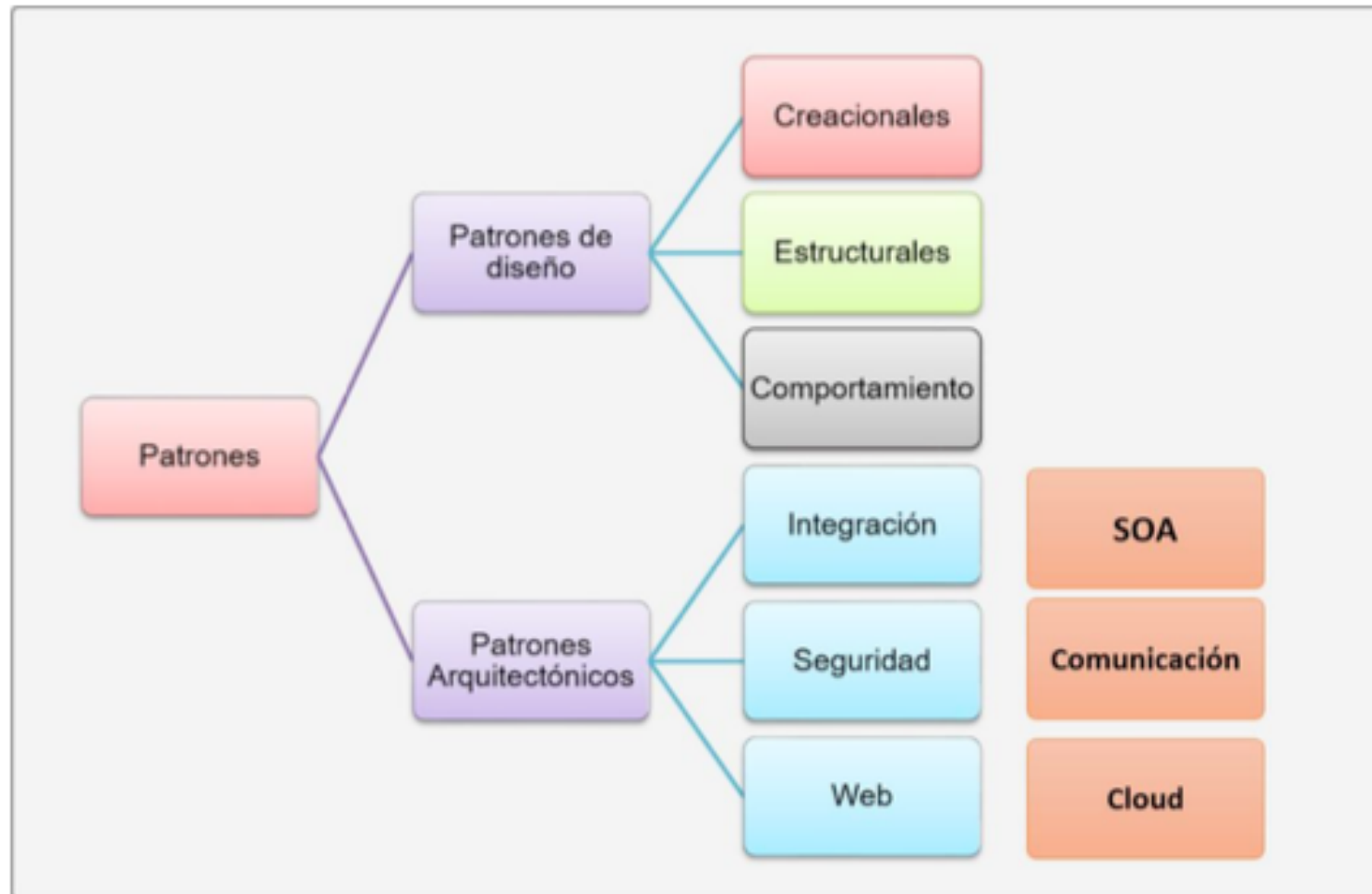


Diferencias entre patrones

Podemos observar que existen diversas definiciones para lo que es un patrón arquitectónico, incluso, hay quienes utilizan el termino patrón para referirse indistintamente a los patrones de diseño y a los patrones arquitectónicos. Sin embargo, queda claro que existe una diferencia fundamental, los patrones de diseño se centran en las clases y los objetos de un componente, es decir, como se crean, estructuran y cómo se comportan en tiempo de ejecución. Por otra parte, los patrones arquitectónicos tiene un alcance más amplio, pues se centra en como los componentes se comportan, se relacionan con los demás y la comunicación que existe entre ellos, aunque además abordan ciertas restricciones tecnológicas, hardware, performance, seguridad, etc.

En otras palabras: “Operan en niveles de abstracción diferentes”.

Tipos de patrones



Hay muchos más patrones arquitectónicos que los indicados en el cuadro.

Pattern Oriented Software Architecture (POSA)



Los patrones de arquitectura expresan el esquema fundamental de organización para sistemas de software. Proveen un conjunto de subsistemas predefinidos, especifican sus responsabilidades e incluyen reglas y guías para organizar las relaciones entre ellos (Microsoft, 2013). POSA como arquitectura orientada a las vistas está compuesta de 4 elementos:

1. La vista **Lógica**, está conformada por los objetos de diseño y el diagrama relacional.
2. La vista de **Proceso**, se encarga de manejar los mecanismos de concurrencia y sincronización de procesos.
3. La Vista de **desarrollo**, hace referencia a la producción de módulos o componentes en un entorno de desarrollo.
4. La vista **física**, relaciona al software con el hardware y otros soportes para artefactos y modelos en entornos distribuidos.

Este modelo coincide con el de Kruchten, pero no hace énfasis en el quinto elemento denominado Vistas de casos de uso.

Estilos arquitectónicos

Otro concepto importante es el de “estilos arquitectónicos” que también se lo suele confundir con el de patrones arquitectónicos, como si fueran sinónimos.

- *“Un estilo arquitectónico define una familia de sistemas en términos de un patrón de organización estructural; Un vocabulario de componentes y conectores, con restricciones sobre cómo se pueden combinar.” - M. Shaw and D. Garlan, *Software architecture: perspectives on an emerging discipline*. Prentice Hall, 1996.*
- *“Un estilo de arquitectura es una asignación de elementos y relaciones entre ellos, junto con un conjunto de reglas y restricciones sobre la forma de usarlo.” - Clements et al., 2003.*
- *Un estilo arquitectónico es una colección con nombre de decisiones de diseño arquitectónico que son aplicables en un contexto de desarrollo dado, restringen decisiones de diseño arquitectónico que son específicas de un sistema particular dentro de ese contexto, y obtienen cualidades beneficiosas en cada uno sistema resultante.” - R. N. Taylor, N. Medvidović and E. M. Dashofy, *Software architecture: Foundations, Theory and Practice*. Wiley, 2009*

Nuevamente observamos que hay varias definiciones de lo que es un estilo arquitectónico, por lo tanto dificulta tener una definición de referencia.

Concepto de estilo arquitectónico



Podemos decir que:

“Un estilo arquitectónico establece un marco de referencia a partir del cual es posible construir sistemas que comparten un conjunto de atributos y características, que hacen posible identificarlos y clasificarlos. “

Los estilos arquitectónicos no existen para resolver un problema de diseño, sino que en su lugar, sirven para nombrar un diseño arquitectónico recurrente.

Diferentes niveles de abstracción



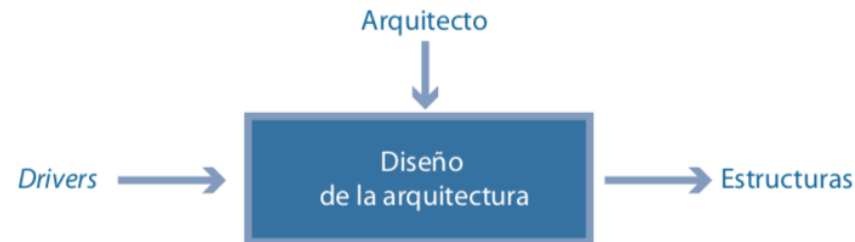
En la figura podemos observar que cada concepto visto opera en diferentes niveles de abstracción y la relación que existe entre ellos. Un estilo arquitectónico puede ser implementado mediante 1 o más patrones arquitectónicos y estos a su vez también pueden ser implementados mediante 1 o más patrones de diseño.



¿Cómo elaboramos el diseño arquitectónico?

Diseño arquitectónico

Principales conceptos:



- ✓ El **diseño arquitectónico** nos permite **definir** cómo debe **organizarse un sistema** y cómo tiene que **diseñarse la estructura global** de esté.
- ✓ Es el **enlace** (puente) entre el **diseño** y la **ingeniería de requisitos**, que identifica los principales componentes estructurales en un sistema y la relación entre ellos.
- ✓ La **salida** del proceso de diseño arquitectónico consiste en un **modelo arquitectónico** que describe la forma en que se organiza el sistema como un conjunto de componentes en comunicación.

Necesidad de seguir un proceso de modelado de la arquitectura



- ▣ La arquitectura es una abstracción de un sistema.
- ▣ Los sistemas pueden tener y tienen una estructura.
- ▣ Todo sistema tiene una arquitectura.
- ▣ Tener una arquitectura no es lo mismo que tener una arquitectura conocida por todos.

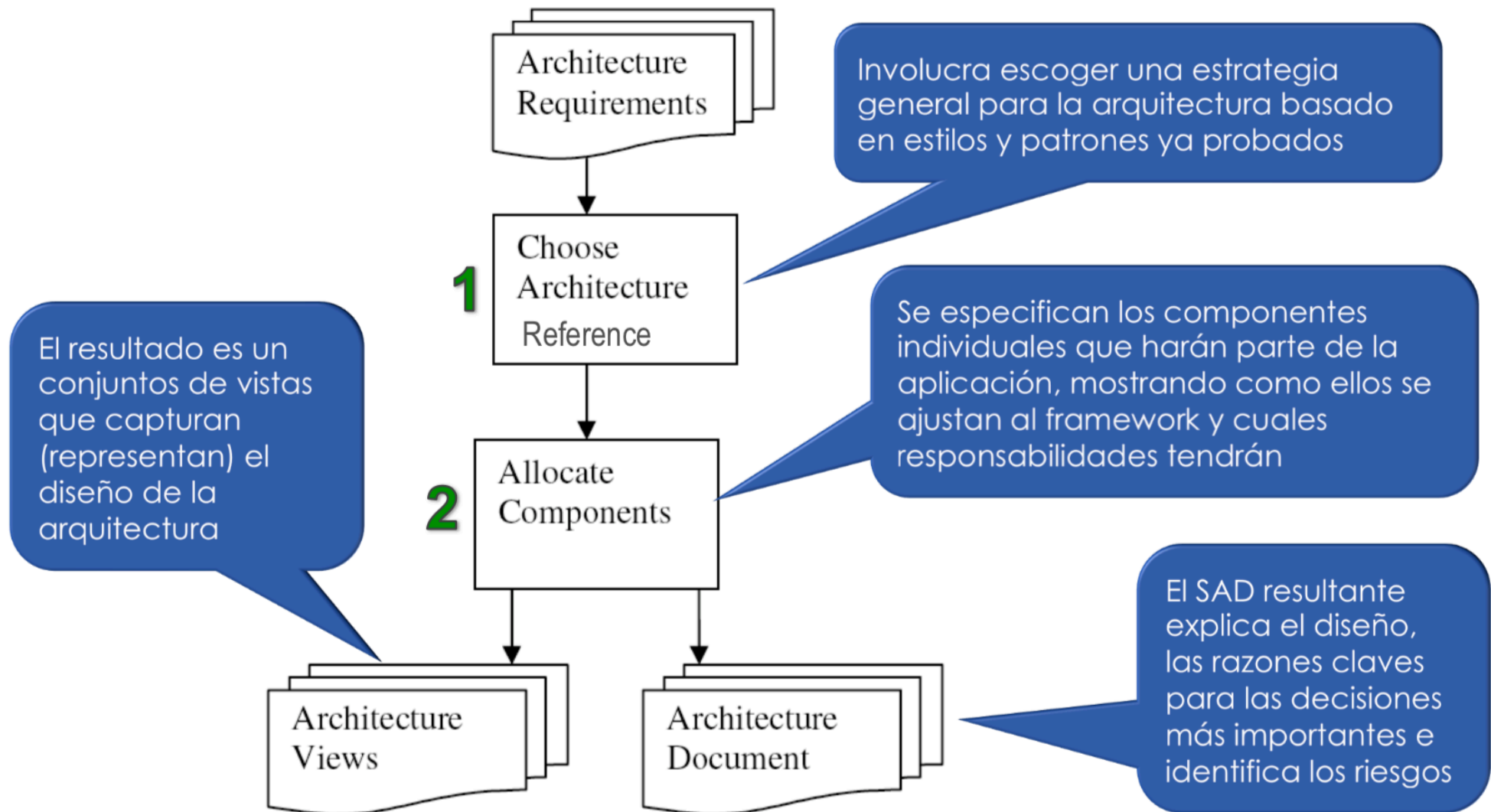
Si no se desarrolla la arquitectura explícitamente, se obtendrá una de todas formas, pero puede no gustarnos lo que obtenemos!

Decisiones en el diseño arquitectónico



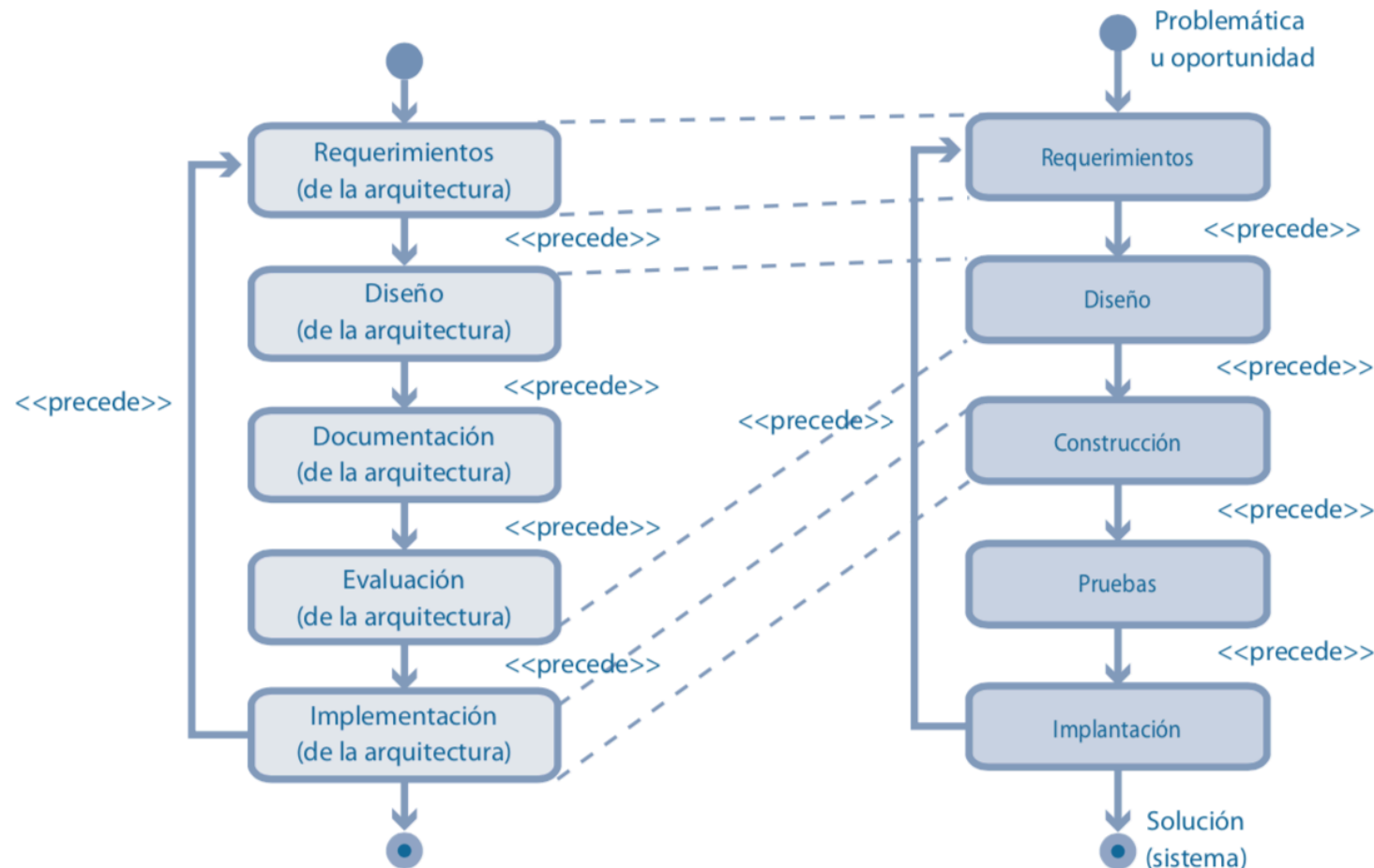
- ¿Existe alguna arquitectura de aplicación genérica que actúe como plantilla para el sistema que se está diseñando?
- ¿Cómo se distribuirá el sistema a través de algunos núcleos o procesadores?
- ¿Qué patrones o estilos arquitectónicos pueden usarse?
- ¿Cuál será el enfoque fundamental usado para estructurar el sistema?
- ¿Cómo los componentes estructurales en el sistema se separarán en subcomponentes?
- ¿Qué estrategia se usará para controlar la operación de los componentes en el sistema?
- ¿Cuál organización arquitectónica es mejor para entregar los requerimientos no funcionales del sistema?
- ¿Cómo se documentará la arquitectura del sistema?

Diseño de la Arquitectura de SW

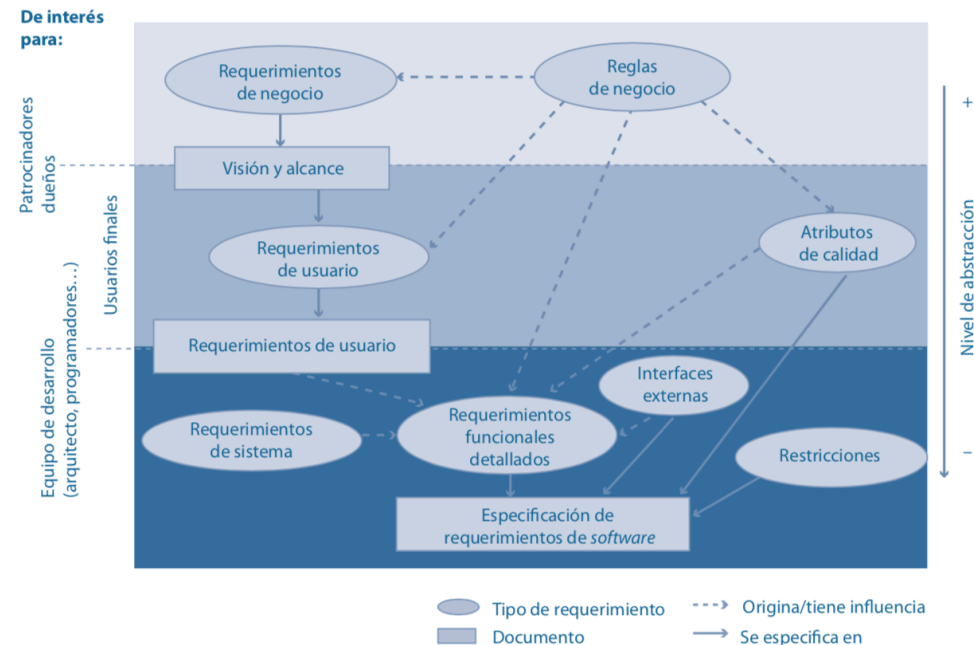


Ciclo de Desarrollo de la Arquitectura

Independiente de la metodología que se utilice para el desarrollo de un sistema software, el desarrollo de la arquitectura tiene como mínimo las siguientes actividades, las cuales se integran con las actividades del desarrollo del sistema.

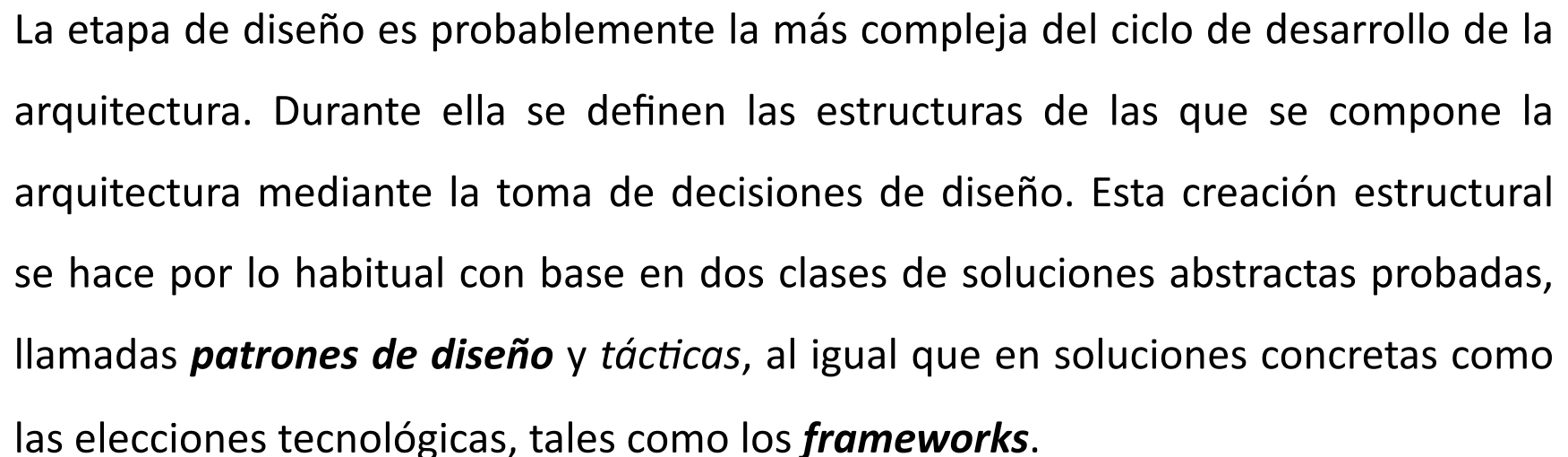


Requisitos de la Arquitectura



Esta etapa se enfoca en los requisitos que influyen sobre la arquitectura (requisitos significativos para la arquitectura de sw).

Como se mencionó, precedentemente, los atributos de calidad juegan un rol preponderante, por lo tanto esta etapa hace énfasis en ellos.



Documentación de la Arquitectura

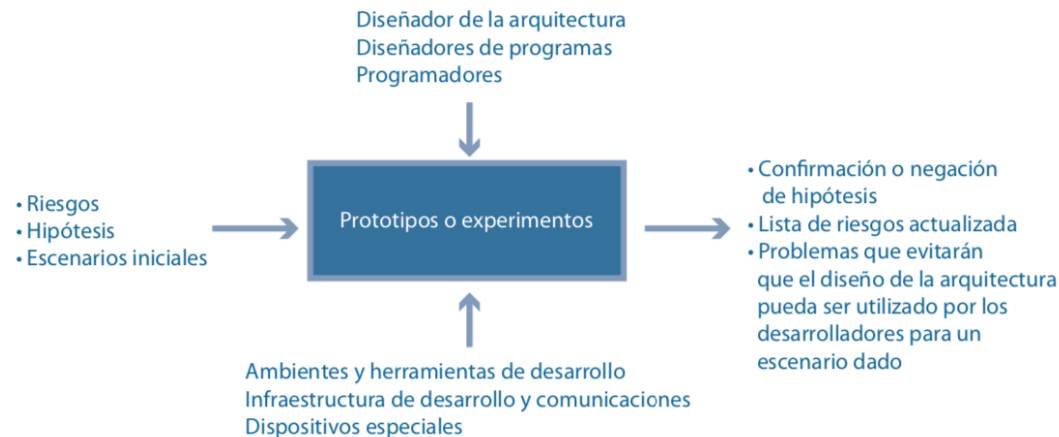


1. Portada
2. Descripción del documento
3. Descripción de proyecto
4. *Drivers arquitectónicos*
5. Contexto del sistema
6. Nivel *n* de descomposición del sistema
 - 6.1 Descripción
 - 6.2 Representación primaria
 - 6.3 Razones de la descomposición
 - 6.4 Catálogos de elementos y relaciones
 - 6.5 Interfaces
7. Mapeo entre perspectivas
8. Índices por palabras, glosarios y acrónimos

Una vez que ha sido creado el diseño de la arquitectura, es necesario darlo a conocer a otros interesados en el sistema, como desarrolladores, responsables de implantación (implementación), líderes de proyecto o el cliente mismo.

La comunicación exitosa depende por lo habitual de que el diseño sea documentado de forma apropiada. A pesar de que durante el diseño se hace una documentación inicial que puede incluir bocetos de las estructuras, o bien capturas de las decisiones de diseño, la documentación formal involucra la representación sus estructuras por medio de *vistas*.

Evaluación de la Arquitectura



Dado que la arquitectura de software juega un rol crucial en el desarrollo, a efecto de identificar posibles riesgos o problemas es conveniente evaluar el diseño una vez que este ha sido documentado.

La ventaja de la evaluación es que representa una actividad que puede realizarse de manera temprana (aun antes de codificar), y que el costo de corrección de los defectos identificados por medio de ella es mucho menor al costo que tendría enmendarlos después de que el sistema ha sido construido.

Implementación de la Arquitectura



Una vez establecida la arquitectura, se construye el sistema.

Durante esta etapa es importante evitar que ocurran desviaciones respecto del diseño definido por el arquitecto.



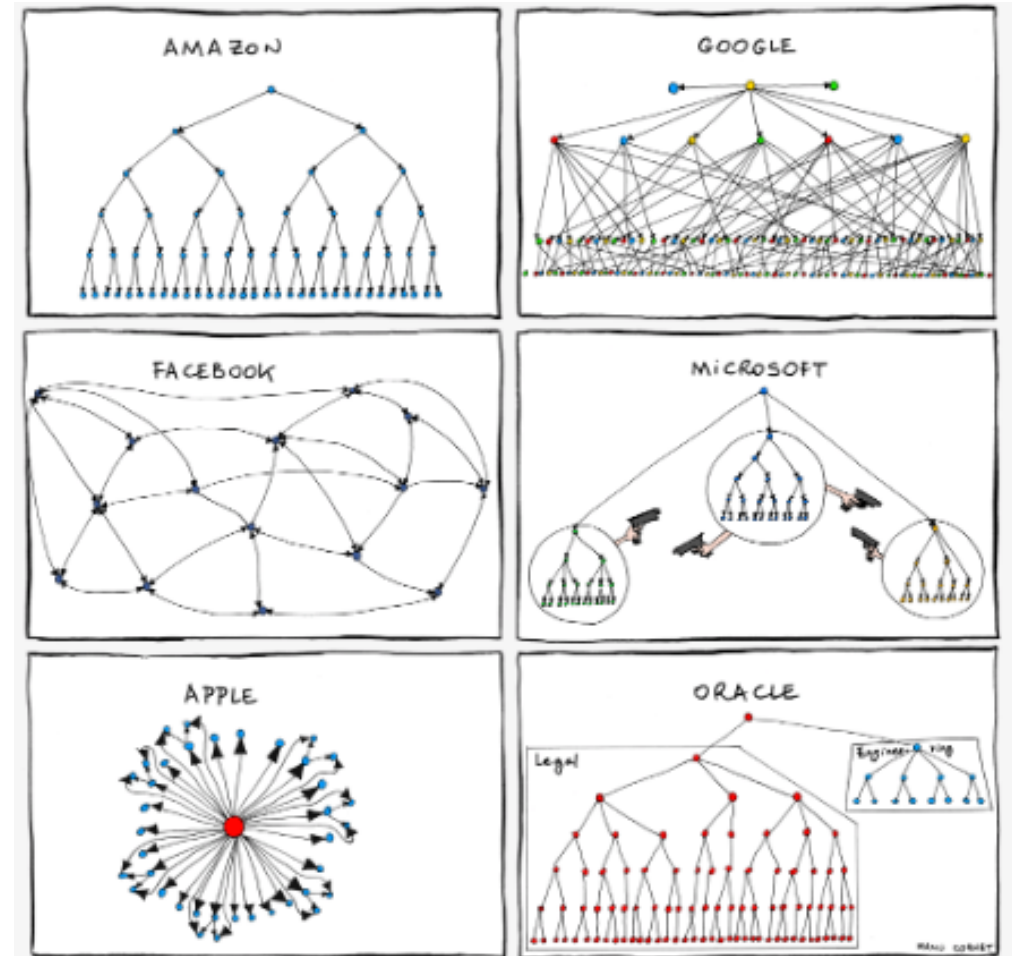


¿Cómo comunicamos la arquitectura?

La importancia de la Comunicación

Ley de Conway:

“Una organización va a poder generar estructuras que imiten la vía de comunicación de su propia organización.”



<http://www.javiergarzas.com/2015/06/conway.html>

En el desarrollo de un sistema de software, el enfoque ágil plantea que los equipos de trabajo sean pequeños y eso se debe a la complejidad de la comunicación cuando el equipo crece o tiene responsabilidades distintas, pasando de una estructura monolítica a otra distribuida. Esto es extensivo a los sistemas desarrollados en la organización.



Comunicar la Arquitectura

“Documentación”

La etapa de documentación se centra en la generación de documentos que describen las estructuras de la arquitectura con el propósito que esta información pueda ser comunicada de manera eficiente a los diversos interesados en el sistema.

Razones para documentar la arquitectura:

1. Mejorar la comunicación de información sobre la arquitectura.
2. Preservar la información sobre la arquitectura.
3. Guiar la generación de artefactos en otras fases del ciclo de vida.
4. Proveer un lenguaje común entre diversos interesados en el sistema.



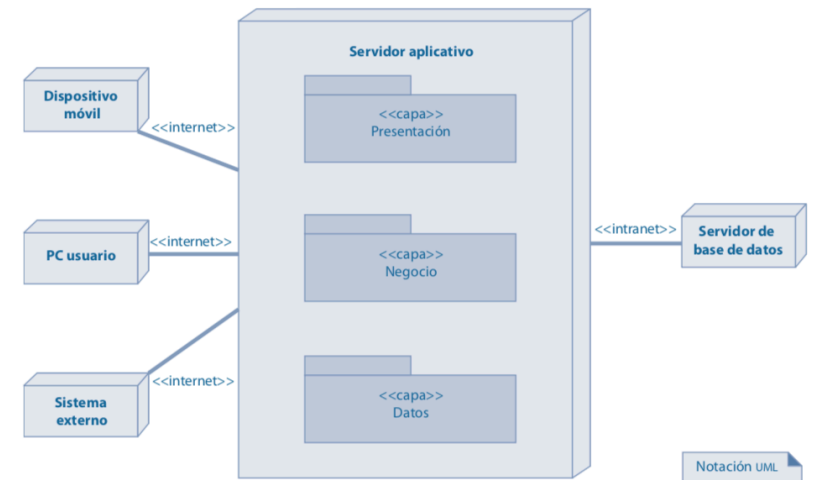
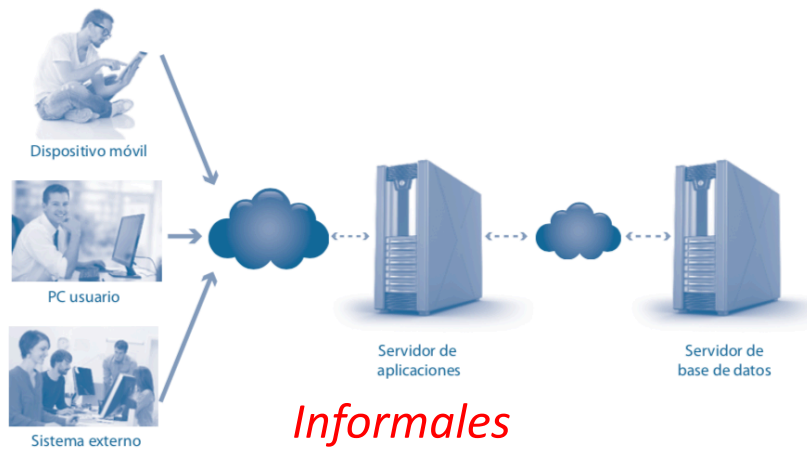
Notaciones

Son un aspecto fundamental para que la documentación cumpla el objetivo de comunicar de manera eficiente información sobre las estructuras con una sintaxis y semántica que minimice, o idealmente erradique, la posibilidad de ambigüedad en su interpretación.

Las podemos clasificar en:

1. Notaciones informales.
2. Notaciones semiformales.
3. Notaciones formales.

Tipos de Notaciones



formales

```

System RedADM = {
    ...

    Component VistaConsultaCorridas = {
        Port p_envia;
        Property codigoFuente: externalFile = "consulta.jsp";
    };

    Component ConsultaCorridasImpl = {
        Port p_recibe;
        Property conexionesMax: integer=100;
        Property latenciaMax: integer= 2;
        Property codigoFuente: externalFile = "consulta.java";
    };

    Connector rmi = {
        Role r_cliente;
        Role r_cliente
    };

    Attachments {
        cliente.envia to rmi.r_cliente;
        servidor.recibe to rmi.r_cliente;
    }
    ...
}
    
```

semi-formales

Vistas de la Arquitectura

Una vista describe una o más estructuras de la arquitectura en términos de los elementos que la conforma

En general una vista se conforma por:

- 1) un diagrama en donde se representan los elementos de la estructura, y
- 2) información textual que ayuda comprender este.

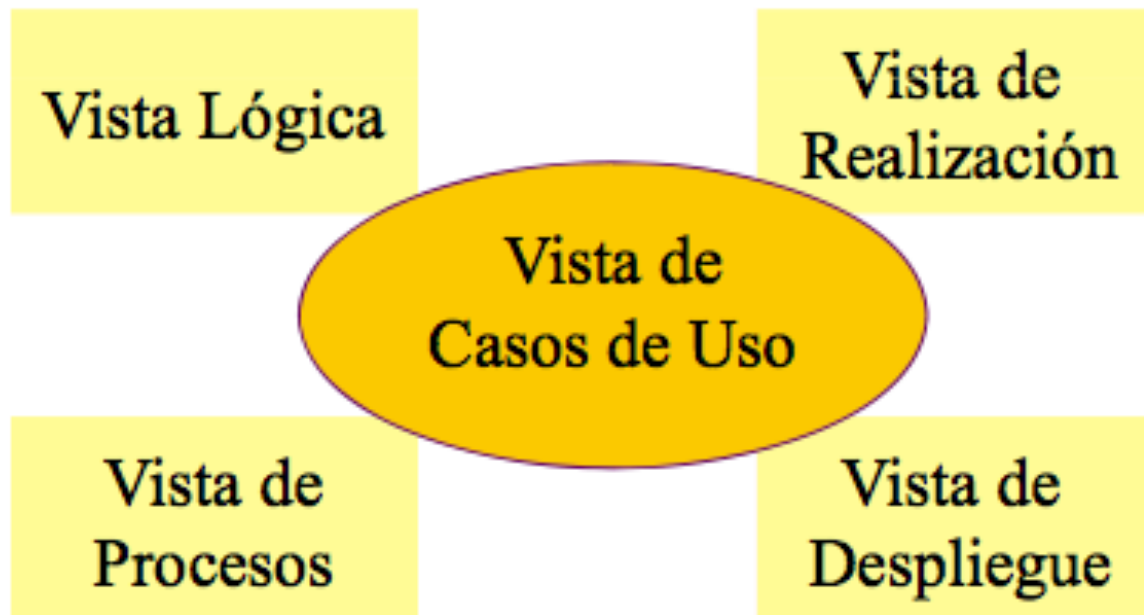
Zachman (Niveles)	TOGAF (Arquitecturas)	4+1 (Vistas)	[BRJ99] (Vistas)	POSA (Vistas)	Microsoft (Vistas)
Scope	Negocios	Lógica	Diseño	Lógica	Lógica
Empresa	Datos	Proceso	Proceso	Proceso	Conceptual
Sistema lógico	Aplicación	Física	Implementación	Física	Física
Tecnología	Tecnología	Desarrollo	Despliegue	Desarrollo	
Representación		Casos de uso	Casos de uso		
Funcionamiento					

Ejemplos de Vistas

- Ejemplos de vistas son:
 - Lógica (requisitos funcionales)
 - De Proceso (conurrencia)
 - Física (distribución)
 - Desarrollo (descomposición del diseño en unidades de implementación)
- Otra clasificación también usada es:
 - Comportamiento
 - Funcional
 - Estructural
 - Modelado de Datos

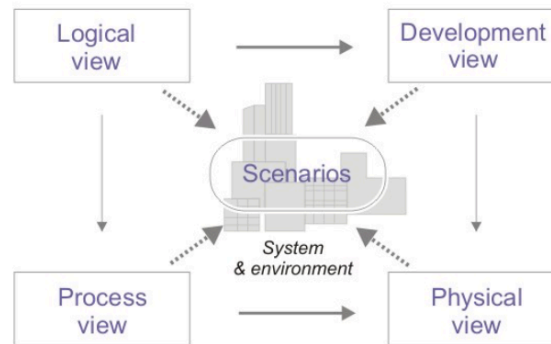
Modelo de arquitectura 4+1 vistas

- Los modelos son instrumentos para visualizar, especificar, construir y documentar la arquitectura del sistema
- Cada vista es una parte de un modelo



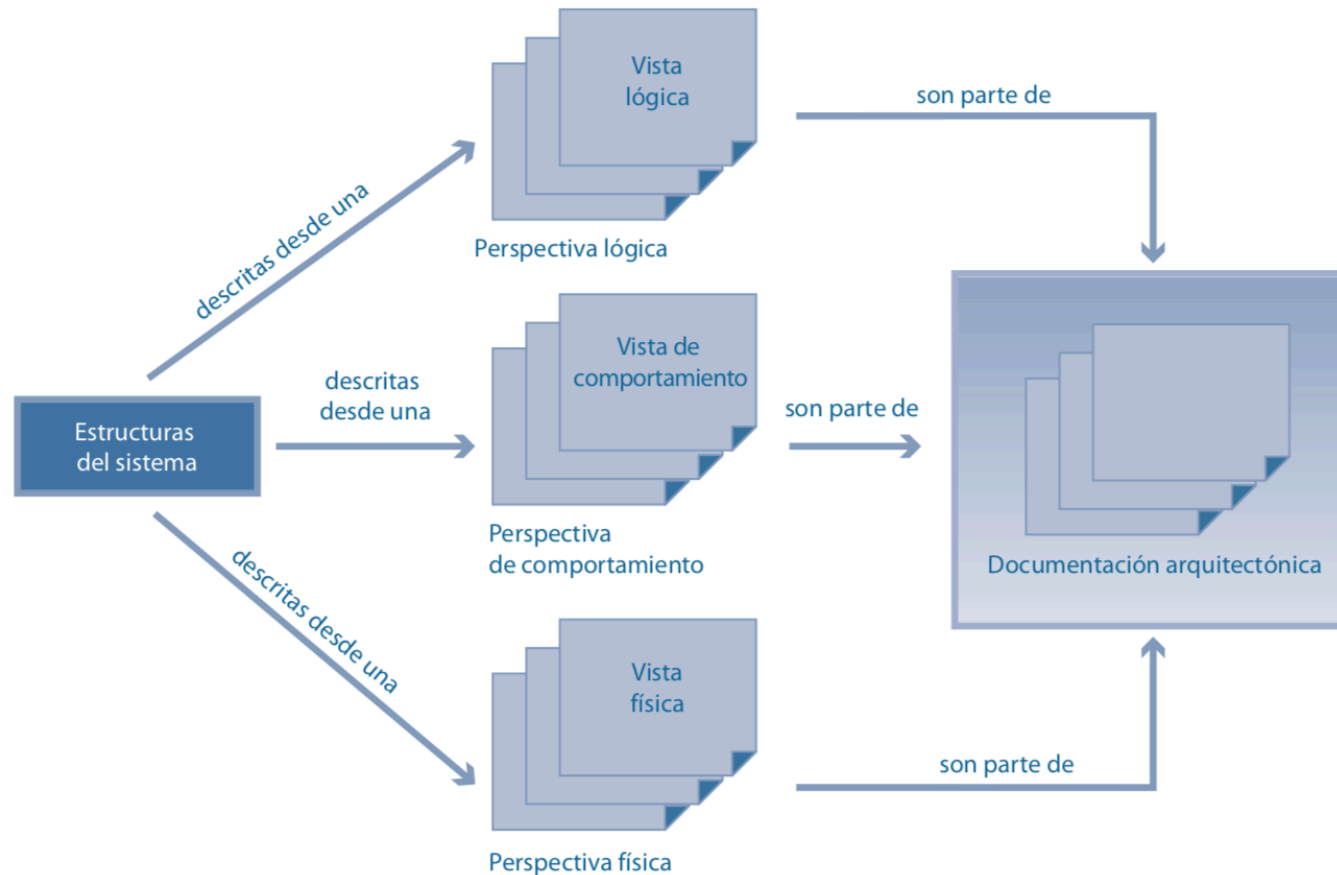
(Philippe Kruchten)

Modelo de arquitectura 4+1 vistas



1. **Vista lógica**, que indique las abstracciones clave en el sistema como objeto o clases de objeto. En este tipo de vista se tienen que relacionar los requerimientos del sistema con entidades.
2. **Vista de proceso**, que muestre cómo, en tiempo de operación, el sistema está compuesto de procesos en interacción. Esta vista es útil para hacer juicios acerca de las características no funcionales del sistema, como el rendimiento y la disponibilidad.
3. **Vista de desarrollo**, que muestre cómo el software está descompuesto para su desarrollo, esto es, indica la descomposición del software en elementos que se implementen mediante un solo desarrollador o equipo de desarrollo, Esta vista es útil para administradores y programadores de software.
4. **Vista física**, que exponga el hardware del sistema y cómo los componentes de software se distribuyen a través de los procesadores en el sistema. Esta vista es útil para los ingenieros de sistemas que planean una implementación de sistema.

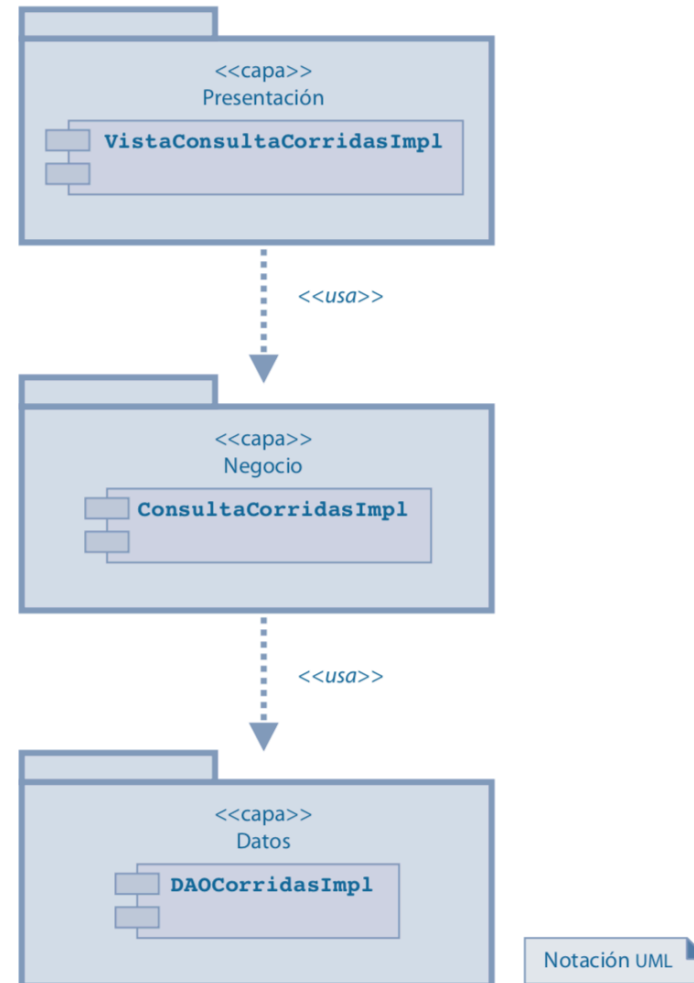
Clasificaciones de vistas



Deben considerarse al menos 3 perspectivas

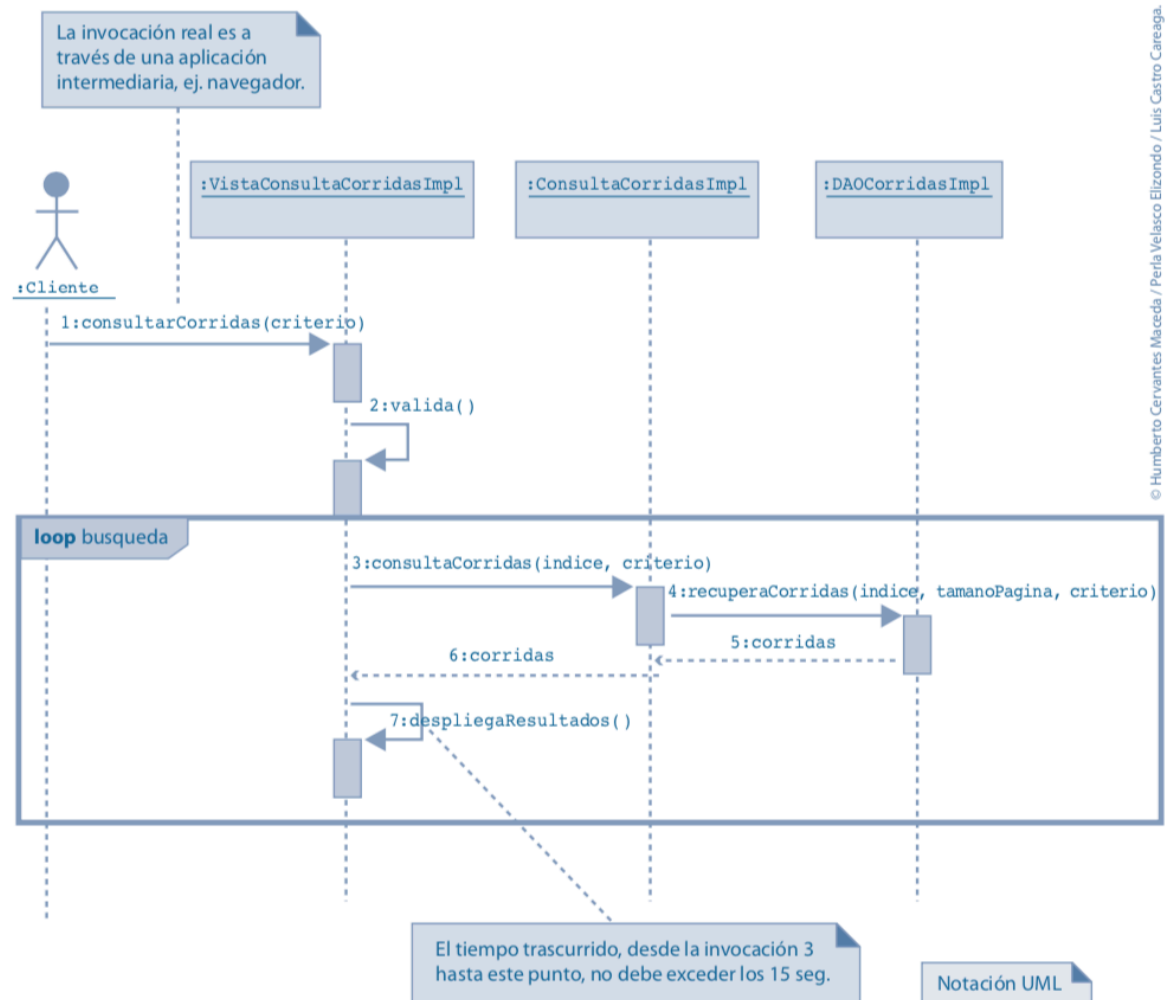
Vistas Lógicas

Estas vistas describen las estructuras arquitectónicas en términos de elementos que toman la forma de unidades de implementación considerando tanto las propiedades como las relaciones u organización de cada una de estas. Las propiedades incluyen aspectos como, por ejemplo, su nombre, las funcionalidades o responsabilidades que le han sido asignadas, las interfaces que define o el lenguaje de programación utilizado para su implementación.



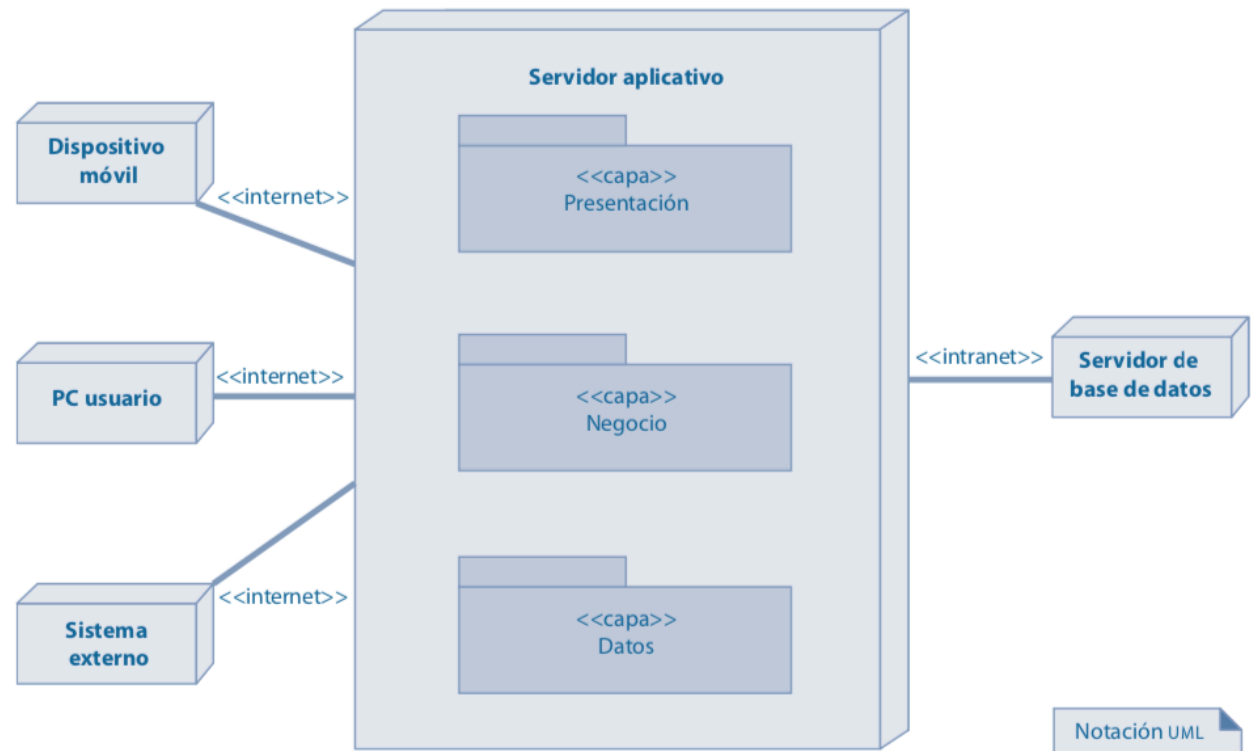
Vistas de comportamiento

Las vistas de comportamiento describen estructuras cuyos elementos denotan entidades visibles en tiempo de ejecución, por ejemplo, *instancias*, *procesos*, *objetos*, *clientes*, *servidores* o *almacenes de datos*.



Vistas físicas

Estas vistas describen estructuras conformadas por elementos “físicos” que mantienen algún tipo de relación con los de las estructuras documentadas en otras vistas. Para una arquitectura es relevante también saber, por ejemplo, cuáles equipos de cómputo albergan las unidades de implementación, cuáles ejecutan las instancias generadas a partir de ellas, y dónde se localizan físicamente estos.



Bibliografía



- Software Architecture. Perspective on an Emerging Discipline. Mary Shaw, David Garlan. Prentice Hall 1996.
- Software Architecture in Practice (SEI Series in Software Engineering). Bass, L., Clements, P., Kasman, R., Bass, K. 3ra. ed. Addison-Wesley Pub Co; 2012.
- Documenting Software Architectures: Views and Beyond. Clements, P. et al. 2da. ed. Addison-Wesley, 2010.
- Evaluating Software Architectures: Methods and Case Studies. Clements, P. et al. 1ra. ed. Addison-Wesley.
- Pattern-Oriented Software Architecture. A System of Patterns. F. Buschman et al. New York, John Wiley and Sons, 1996.
- Pattern-Oriented Software Architecture. Patterns for Concurrent and Networked Objects. Volumen 2. Douglas Schmith, Michael Stal, Hans Rohnert, Frank Buschman. John Wiley and Sons, 2000.
- Krutchen, P. (1995). "The 4+1 view model of software architecture". IEEE Software.
- Bass, L., Clements, P. y Kazman, R. (2003). *Software Architecture in Practice, 2a ed. Boston: Addison-Wesley.*



UNIVERSIDAD CHAMPAGNAT



www.uch.edu.ar



[uch.argentina](https://www.facebook.com/uch.argentina)



[@uchampagnat](https://twitter.com/uchampagnat)

CAMPUS CENTRAL

Belgrano 721 - Godoy Cruz, Mendoza - C.P.5501 - Argentina.
Teléfono 0261-4248443

CAMPUS SAN RAFAEL

Libertador 150 - San Rafael - Mendoza - C.P.5600 - Argentina.
Teléfono 0260-4424812

CAMPUS VALLE DE UCO

Pueyrredón 58, 1º Piso - Tunuyán - Mendoza - C.P.5560 - Argentina
Teléfono 02622-460644